

第一节课

NLP 基础与文本预处理

完整教学内容 · 详细实践指南



课程时长	难度等级	编程语言
4 小时	初学者	Python

课程目标： 从零开始掌握自然语言处理的基础知识与核心技能

适用对象： 具备 Python 基础的 NLP 初学者

先修要求： Python 基础编程、基本数学统计知识

2025 年 9 月 5 日

目录

1 课程概览与学习目标	3
1.1 课程结构与时间安排	4
2 第一部分：NLP 理论基础 (90 分钟)	4
2.1 1.1 自然语言处理概述 (30 分钟)	4
2.1.1 NLP 技术发展历程	5
2.1.2 NLP 核心应用领域	7
2.1.3 NLP 面临的核心挑战	8
2.2 1.2 NLP 任务体系架构 (30 分钟)	9
2.2.1 任务层次结构分析	9
2.2.2 词法分析层详解	10
2.2.3 句法分析层详解	11
2.2.4 语义分析层详解	12
2.3 1.3 文本表示方法详解 (30 分钟)	13
2.3.1 传统符号化表示方法	13
2.3.2 N-gram 语言模型	17
2.3.3 现代分布式表示简介	19
3 第二部分：开发环境搭建 (30 分钟)	22
3.1 2.1 Python 环境配置 (15 分钟)	22
3.2 2.2 核心库功能详解	26
3.3 2.3 Jupyter Notebook 高效使用指南 (15 分钟)	28
4 第三部分：文本预处理实践 (120 分钟)	29
4.1 3.1 基础文本处理 (40 分钟)	29
4.1.1 文本清洗技术详解	29
4.1.2 中文分词技术深度解析	35
4.2 3.2 停用词处理与词频统计 (40 分钟)	43
4.3 3.3 综合文本处理流水线 (40 分钟)	43
4.3.1 端到端文本处理系统	43
4.3.2 停用词管理系统	54
4.3.3 深度词频分析系统	64
5 课后练习设计	82
5.0.1 练习 1: 基础文本清洗项目 ()	85

5.0.2	练习 2: 词频分析与可视化 ()	87
5.0.3	练习 3: 智能停用词优化系统 ()	89
5.0.4	练习 4: 领域文本分析综合项目 ()	91
6	技术栈概览	92
7	Python 代码示例	92
8	总结与展望	109
8.1	课程总结	109
8.2	后续课程展望	109
8.3	学习资源推荐	111
8.4	开发工具推荐	111
9	附录	111
9.1	技术参考资料	111
9.2	常见问题解答 (FAQ)	111
9.3	持续学习建议	114

1 课程概览与学习目标

课程学习目标

知识目标 (Knowledge Objectives)

- 理解 NLP 核心概念：掌握自然语言处理的定义、发展历程和主要应用领域
- 掌握文本表示方法：深入理解从符号化到分布式表示的技术演进
- 熟悉中文 NLP 特性：了解中文文本处理的独特挑战和解决方案
- 学会文本预处理技术：掌握完整的文本清洗、分词和标准化流程

技能目标 (Skill Objectives)

- 环境搭建能力：独立配置 Python NLP 开发环境和相关工具包
- 代码实现能力：编写高质量、可维护的文本处理代码
- 数据分析能力：进行深入的词频统计分析和数据可视化
- 问题解决能力：诊断文本处理问题并提出优化解决方案

素养目标 (Competency Objectives)

- 数据驱动思维：建立基于数据分析进行决策的科学思维方式
- 系统化思维：学会构建完整、规范的文本处理流水线系统
- 质量意识：培养对数据质量和处理效果的敏感性和判断力
- 持续学习能力：掌握跟踪 NLP 技术发展和学习最佳实践的方法

1.1 课程结构与时间安排

表 1: 详细课程时间分配表

阶段	内容模块	时长	教学方式
理论基础	NLP 概述与发展历程	30 分钟	讲授 + 案例分析
	NLP 任务体系架构	30 分钟	交互演示 + 讨论
	文本表示方法详解	30 分钟	数学推导 + 可视化
环境搭建	Python 环境配置	15 分钟	实操演示
	Jupyter 使用技巧	15 分钟	实践指导
实践编程	基础文本处理	40 分钟	编程实战
	停用词与词频统计	40 分钟	代码讲解 + 练习
	综合处理流水线	40 分钟	项目实践
课后练习	作业布置与答疑	30 分钟	指导 + 讨论

2 第一部分：NLP 理论基础 (90 分钟)

2.1 1.1 自然语言处理概述 (30 分钟)

什么是自然语言处理？

定义： 自然语言处理（Natural Language Processing, NLP）是计算机科学、人工智能和语言学的交叉学科，致力于研究能实现人与计算机之间用自然语言进行有效通信的各种理论和方法。

核心目标： 让计算机能够理解、解释、操作和生成人类语言，从而在各种任务中表现出类似人类的语言能力。

学科交叉性：

- **计算机科学：** 算法设计、数据结构、机器学习
- **语言学：** 语音学、语法学、语义学、语用学
- **数学：** 概率论、统计学、线性代数、信息论
- **认知科学：** 心理学、认知心理学、神经科学

2.1.1 NLP 技术发展历程

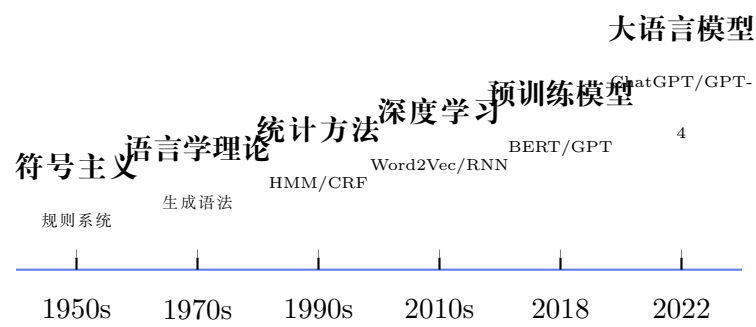


图 1: NLP 技术发展时间轴

各阶段技术特点详解

1. 符号主义时代 (1950s-1960s)

- 基于专家规则的系统设计
- 代表：Georgetown 机器翻译实验 (1954)
- 局限：规则复杂度指数增长，覆盖率低

2. 语言学理论时代 (1970s-1980s)

- 乔姆斯基生成语法理论的应用
- 语法解析器和句法分析系统
- 局限：缺乏语义理解，实用性有限

3. 统计方法时代 (1990s-2000s)

- 基于大规模语料库的统计建模
- 隐马尔可夫模型 (HMM)、条件随机场 (CRF)
- 突破：数据驱动，自动学习模式

4. 深度学习时代 (2010s)

- Word2Vec 词向量表示学习
- 循环神经网络 (RNN/LSTM/GRU)
- 注意力机制 (Attention Mechanism)

5. 预训练模型时代 (2018-present)

- Transformer 架构革命
- BERT 双向编码器表示
- GPT 系列生成式预训练模型
- 大规模预训练 + 下游任务微调范式

2.1.2 NLP 核心应用领域

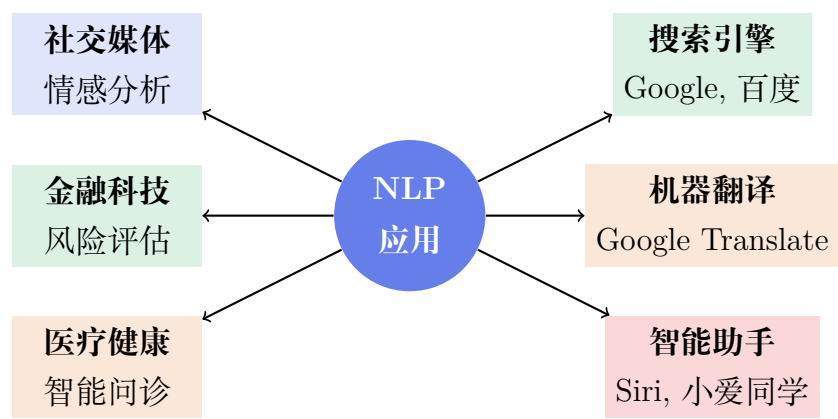


图 2: NLP 主要应用领域分布图

表 2: NLP 应用领域详细分析

应用领域	核心技术	典型产品	技术挑战
搜索引擎	查询理解、文档检索、相关性排序	Google、百度、Bing	查询意图理解、多模态检索
机器翻译	序列到序列学习、注意力机制	Google Translate、DeepL	低资源语言、文化语境
智能助手	语音识别、对话系统、任务规划	Siri、Alexa、小爱同学	多轮对话、常识推理
社交媒体	情感分析、内容审核、推荐系统	微博、Facebook、抖音	讽刺检测、网络用语
金融科技	文本挖掘、风险评估、舆情监控	蚂蚁金服、平安科技	专业术语、实时性要求
医疗健康	病历分析、药物发现、诊断辅助	IBM Watson、科大讯飞	医学专业性、安全性要求

2.1.3 NLP 面临的核心挑战

语言理解的根本困难

1. 歧义性问题 (Ambiguity)

- **词汇歧义**：一个词有多种含义
 - 例：“苹果公司发布新产品” vs “这个苹果很甜”
 - 解决方案：词义消歧算法、上下文建模
- **句法歧义**：句子结构有多种解析方式
 - 例：“他看见了山上的老人用望远镜”
 - 解决方案：概率文法、依存句法分析
- **语义歧义**：句子语义有多种理解
 - 例：“银行”指金融机构还是河岸？
 - 解决方案：知识图谱、语义表示学习

2. 上下文依赖性 (Context Dependency)

- **指代消解**：代词指向的具体对象
- **省略现象**：省略的成分需要从上下文推断
- **话语连贯性**：句子间的逻辑关系

3. 语言动态性 (Language Dynamics)

- **新词涌现**：“打 call”、“躺平”、“内卷”
- **语义演化**：词汇含义随时间变化
- **方言变体**：地域性语言差异

4. 隐含知识依赖 (Implicit Knowledge)

- **常识推理**：“春节回家”蕴含的文化背景
- **背景知识**：专业领域的先验知识
- **社会文化**：语言使用的社会语境

2.2 1.2 NLP 任务体系架构 (30 分钟)

2.2.1 任务层次结构分析

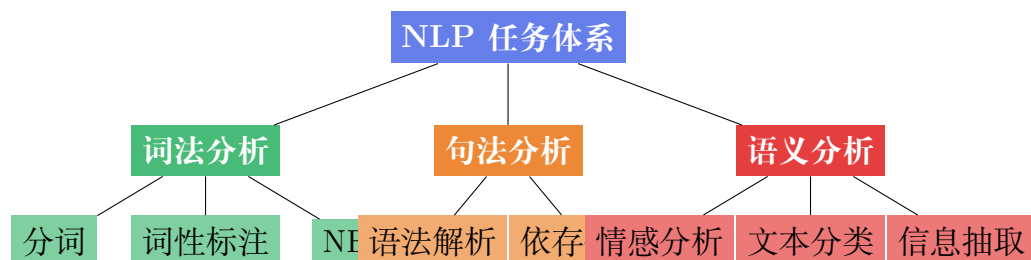


图 3: NLP 任务层次结构树

2.2.2 词法分析层详解

词法分析 (Lexical Analysis)

定义：对文本进行词汇层面的分析，是 NLP 处理的基础环节。

1. 分词 (Tokenization)

- **目标：**将连续的文本序列切分成有意义的词汇单元
- **中文特点：**没有天然的词边界，需要算法识别
- **常用方法：**
 - 基于词典的最大匹配算法
 - 基于统计的 N-gram 模型
 - 基于机器学习的序列标注

2. 词性标注 (Part-of-Speech Tagging)

- **目标：**为每个词汇标注其语法角色
- **标注体系：**名词 (n)、动词 (v)、形容词 (a)、副词 (d) 等
- **技术挑战：**一词多性、歧义消解

3. 命名实体识别 (Named Entity Recognition, NER)

- **目标：**识别文本中的专有名词实体
- **实体类型：**人名 (PER)、地名 (LOC)、机构名 (ORG)、时间 (TIME) 等
- **标注格式：**BIO 标注、BILOU 标注

4. 词干提取与词形还原

- **词干提取：**去除词缀，保留词根
- **词形还原：**将词汇还原为字典形式
- **中文特点：**形态变化相对简单

表 3: 词法分析任务示例

原始文本	分词结果	词性标注	命名实体
张三在北京大学 学习自然语言处 理	张三/在/北京大学/	张三/nr 在/p	张三/PER
	学习/自然语言处理	北京大学/nt	北京大学/ORG
		学习/v 自然语言处理/n	自然语言处理/TERM

2.2.3 句法分析层详解

句法分析 (Syntactic Analysis)

定义：分析句子的语法结构，理解词语间的语法关系。

1. 成分句法分析 (Constituency Parsing)

• 目标：构建句子的短语结构树

• 基本概念：短语 (Phrase)、成分 (Constituent)

• 文法形式：上下文无关文法 (CFG)

2. 依存句法分析 (Dependency Parsing)

• 目标：分析词语间的依存关系

• 核心概念：中心词 (Head)、修饰词 (Dependent)

• 关系类型：主谓关系、动宾关系、定语关系等



图 4: 两种句法分析方法对比

2.2.4 语义分析层详解

语义分析 (Semantic Analysis)

定义：理解文本的含义，是 NLP 的高层任务。

1. 词义消歧 (Word Sense Disambiguation)

- **目标：**确定多义词在特定上下文中的具体含义
- **方法：**基于词典、基于语料库、基于知识图谱
- **评估：**精确率、召回率、F1 分数

2. 语义角色标注 (Semantic Role Labeling)

- **目标：**识别句子中谁对谁做了什么
- **角色类型：**施事者 (Agent)、受事者 (Patient)、工具 (Instrument)
- **标注体系：**PropBank、FrameNet

3. 情感分析 (Sentiment Analysis)

- **粒度层次：**文档级、句子级、属性级
- **情感维度：**极性分类、情感强度、情感类别
- **挑战：**讽刺识别、隐含情感、领域适应

4. 文本分类 (Text Classification)

- **任务类型：**主题分类、垃圾邮件检测、意图识别
- **方法演进：**朴素贝叶斯 → SVM → 深度学习 → 预训练模型
- **评估指标：**准确率、精确率、召回率、F1 分数、AUC

表 4: NLP 任务技术难度与应用广度对比

任务类别	技术难度	应用广度	数据需求	计算复杂度	商业价值
分词			中等	低	基础设施
词性标注			中等	低	辅助分析
命名实体识别			高	中等	信息抽取
句法分析			高	中等	语法检查
情感分析			中等	中等	舆情监控
文本分类			中等	中等	内容审核
机器翻译			极高	高	跨语言交流
问答系统			极高	高	智能客服

2.3 1.3 文本表示方法详解 (30 分钟)

2.3.1 传统符号化表示方法

One-hot 编码

基本思想： 用一个维度为词汇表大小的向量表示每个词，该词对应位置为 1，其余位置为 0。

数学定义： 对于词汇表 $V = \{w_1, w_2, \dots, w_{|V|}\}$ ，词汇 w_i 的 one-hot 表示为：

$$\text{OneHot}(w_i) = [0, 0, \dots, \underbrace{1}_{i\text{-th position}}, \dots, 0] \in \mathbb{R}^{|V|}$$

(1)

优点：

- 简单直观，易于理解和实现
- 词汇表示完全独立，避免语义混淆
- 计算简单，适合小规模词汇表

缺点：

- 维度灾难：词汇表大小通常为 10k-100k
- 语义鸿沟：无法表示词汇间的语义关系
- 稀疏性：向量中只有一个非零元素

One-hot 编码实例

假设词汇表为： $V = \{ \text{“我”}, \text{“爱”}, \text{“自然”}, \text{“语言”}, \text{“处理”} \}$
各词的 one-hot 表示：

$$\text{“我”} \rightarrow [1, 0, 0, 0, 0] \quad (2)$$

$$\text{“爱”} \rightarrow [0, 1, 0, 0, 0] \quad (3)$$

$$\text{“自然”} \rightarrow [0, 0, 1, 0, 0] \quad (4)$$

$$\text{“语言”} \rightarrow [0, 0, 0, 1, 0] \quad (5)$$

$$\text{“处理”} \rightarrow [0, 0, 0, 0, 1] \quad (6)$$

问题分析：“自然”和“语言”在语义上高度相关，但它们的 one-hot 向量点积为 0，无法体现语义相似性。

词袋模型 (Bag of Words, BoW)

核心思想：忽略文本中词汇的顺序信息，只考虑词汇的出现频率。

数学表示：对于文档 d 和词汇表 V ，词袋表示为：

$$\text{BoW}(d) = [c_1, c_2, \dots, c_{|V|}] \quad (7)$$

其中 c_i 表示词汇 w_i 在文档 d 中的出现次数。

变体形式：

- **二元词袋：**只记录词汇是否出现 (0 或 1)
- **标准化词袋：**词频除以文档总词数
- **对数词袋：** $\log(1 + c_i)$ 来减少高频词的影响

实现算法：

Algorithm 1 词袋模型构建算法

输入：文档集合 $D = \{d_1, d_2, \dots, d_n\}$

输出：词袋矩阵 $M \in \mathbb{R}^{n \times |V|}$

1. 构建词汇表: $V = \bigcup_{i=1}^n \text{words}(d_i)$
 2. **for** each document d_i **do**
 3. **for** each word $w_j \in V$ **do**
 4. $M[i][j] = \text{count}(w_j, d_i)$
 5. **end for**
 6. **end for**
 7. **return** M
-

TF-IDF 权重机制

动机：词袋模型忽略了词汇的重要性差异，需要一种机制来突出重要词汇。

术语频率 (Term Frequency, TF)：

$$\text{TF}(t, d) = \frac{\text{count}(t, d)}{\sum_k \text{count}(k, d)} \quad (8)$$

逆文档频率 (Inverse Document Frequency, IDF)：

$$\text{IDF}(t) = \log \left(\frac{|D|}{|\{d \in D : t \in d\}|} \right) \quad (9)$$

TF-IDF 权重：

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t) \quad (10)$$

直观理解：

- **TF：**词汇在当前文档中的重要性
- **IDF：**词汇在整个语料库中的稀有程度
- **TF-IDF：**平衡局部重要性和全局稀有性

变体公式：

- 对数 TF: $\text{TF}(t, d) = \log(1 + \text{count}(t, d))$
- 平滑 IDF: $\text{IDF}(t) = \log \left(\frac{|D|+1}{|\{d \in D : t \in d\}|+1} \right)$
- L2 标准化: $\frac{\text{TF-IDF}(t, d)}{\|\text{TF-IDF}(d)\|_2}$

表 5: TF-IDF 计算示例

文档/词汇	” 机器”	” 学习”	” 算法”	” 的”	” 是”
文档 1: ” 机器学习算法”	1	1	1	0	0
文档 2: ” 学习是重要的”	0	1	0	1	1
文档 3: ” 算法的设计”	0	0	1	1	0
DF	1	2	2	2	1
IDF	1.099	0.405	0.405	0.405	1.099

对于文档 1 中的词汇” 机器”：

$$\text{TF}(\text{” 机器”}, d_1) = \frac{1}{3} = 0.333 \quad (11)$$

$$\text{IDF}(\text{” 机器”}) = \log\left(\frac{3}{1}\right) = 1.099 \quad (12)$$

$$\text{TF-IDF}(\text{” 机器”}, d_1) = 0.333 \times 1.099 = 0.366 \quad (13)$$

2.3.2 N-gram 语言模型

N-gram 模型原理

基本思想： 利用前 n-1 个词来预测当前词，基于马尔可夫假设。

马尔可夫假设： 当前词只依赖于前面有限个词，即：

$$P(w_i | w_1, w_2, \dots, w_{i-1}) \approx P(w_i | w_{i-n+1}, \dots, w_{i-1}) \quad (14)$$

不同 N 值的模型：

- **Unigram (N=1):** $P(w_i) = \frac{\text{count}(w_i)}{N}$
- **Bigram (N=2):** $P(w_i | w_{i-1}) = \frac{\text{count}(w_{i-1}w_i)}{\text{count}(w_{i-1})}$
- **Trigram (N=3):** $P(w_i | w_{i-2}w_{i-1}) = \frac{\text{count}(w_{i-2}w_{i-1}w_i)}{\text{count}(w_{i-2}w_{i-1})}$

句子概率计算：

$$P(S) = P(w_1, w_2, \dots, w_m) = \prod_{i=1}^m P(w_i | w_{i-n+1}, \dots, w_{i-1}) \quad (15)$$

平滑技术（解决数据稀疏问题）：

- **拉普拉斯平滑：** $P(w_i | w_{i-1}) = \frac{\text{count}(w_{i-1}w_i) + 1}{\text{count}(w_{i-1}) + |V|}$
- **Good-Turing 平滑：** 基于频率的频率进行估计
- **Katz 回退：** 高阶 N-gram 回退到低阶 N-gram

N-gram 模型计算示例

给定语料：“我爱自然语言处理”，“我学习机器学习”，“自然语言很有趣”

Bigram 统计：

- $\text{count}(\text{“我”}, \text{“爱”}) = 1$
- $\text{count}(\text{“我”}, \text{“学习”}) = 1$
- $\text{count}(\text{“自然”}, \text{“语言”}) = 2$
- $\text{count}(\text{“语言”}, \text{“处理”}) = 1$
- $\text{count}(\text{“语言”}, \text{“很”}) = 1$

概率计算：

$$P(\text{“爱”}|\text{“我”}) = \frac{\text{count}(\text{“我爱”})}{\text{count}(\text{“我”})} = \frac{1}{2} = 0.5 \quad (16)$$

$$P(\text{“学习”}|\text{“我”}) = \frac{\text{count}(\text{“我学习”})}{\text{count}(\text{“我”})} = \frac{1}{2} = 0.5 \quad (17)$$

$$P(\text{“语言”}|\text{“自然”}) = \frac{\text{count}(\text{“自然语言”})}{\text{count}(\text{“自然”})} = \frac{2}{2} = 1.0 \quad (18)$$

句子“我爱自然语言”的概率：

$$P(\text{“我爱自然语言”}) = P(\text{“我”}) \times P(\text{“爱”}|\text{“我”}) \times P(\text{“自然”}|\text{“爱”}) \times P(\text{“语言”}|\text{“自然”}) \quad (19)$$

$$= 0.2 \times 0.5 \times 1.0 \times 1.0 = 0.1 \quad (20)$$

2.3.3 现代分布式表示简介

词嵌入 (Word Embeddings)

核心理念：将词汇表示为低维稠密向量，捕获语义相似性。

分布假说：“相似上下文中出现的词汇具有相似的含义”——Firth (1957)

数学表示：每个词汇 w_i 映射为一个 d 维向量：

$$w_i \rightarrow \mathbf{v}_i \in \mathbb{R}^d \quad \text{其中 } d \ll |V| \quad (21)$$

语义相似性：通过向量距离度量（余弦相似度、欧几里得距离）

主要方法：

- Word2Vec: Skip-gram 与 CBOW 模型
- GloVe: 全局向量表示
- FastText: 考虑子词信息
- ELMo: 上下文相关表示
- BERT: 双向编码器表示

经典例子：向量运算

$$\text{vec}(\text{“国王”}) - \text{vec}(\text{“男人”}) + \text{vec}(\text{“女人”}) \approx \text{vec}(\text{“女王”}) \quad (22)$$

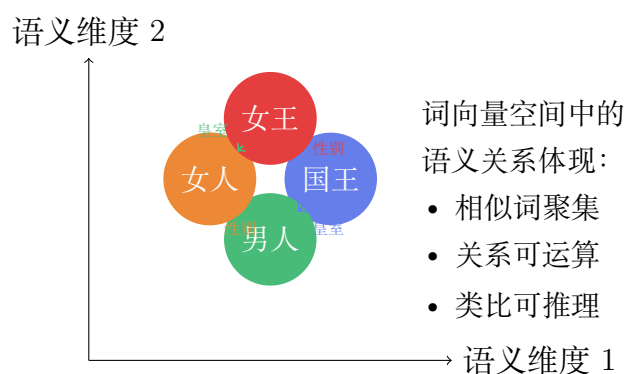


图 5: 词向量空间中的语义关系示意图

表 6: 传统表示方法与现代表示方法对比

表示方法	向量维度	稠密度	语义信息	计算复杂度	内存占用
One-hot	$ V $ (10k-100k)	稀疏	无	$O(V)$	高
BoW	$ V $ (10k-100k)	稀疏	弱	$O(V)$	高
TF-IDF	$ V $ (10k-100k)	稀疏	中等	$O(V)$	高
Word2Vec	50-300	稠密	强	$O(d)$	低
BERT	768/1024	稠密	很强	$O(d)$	中等

3 第二部分：开发环境搭建 (30 分钟)

3.1 2.1 Python 环境配置 (15 分钟)

环境搭建完整指南

第一步：安装 Anaconda

- 下载地址：<https://www.anaconda.com/products/distribution>
- 选择 Python 3.8+ 版本
- 验证安装：打开命令行输入 `conda --version`

第二步：创建虚拟环境

```
# 创建名为 nlp_course 的虚拟环境
conda create -n nlp_course python=3.8

# 激活环境
conda activate nlp_course

# 验证环境
python --version
```

Listing 1: 创建 NLP 专用环境

第三步：安装核心库

```
# 基础数据处理库
pip install numpy==1.21.0
pip install pandas==1.3.0

# 可视化库
pip install matplotlib==3.4.2
pip install seaborn==0.11.1

# 中文 NLP 库
pip install jieba==0.42.1

# 机器学习库
pip install scikit-learn==0.24.2

# 开发环境
pip install jupyter==1.0.0
pip install ipykernel

# 其他工具库
```

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
NLP课程环境验证脚本
功能：检查所需库是否正确安装
"""

import sys
import importlib

def check_package(package_name, alias=None):
    """检查包是否能正常导入"""
    try:
        if alias:
            globals()[alias] = importlib.import_module(package_name)
        else:
            importlib.import_module(package_name)
        print(f" {package_name} - 安装成功")
        return True
    except ImportError as e:
        print(f" {package_name} - 安装失败: {e}")
        return False

def main():
    """主函数：执行环境检查"""
    print("=" * 50)
    print("NLP课程环境检查")
    print("=" * 50)

    # 检查Python版本
    python_version = sys.version_info
    print(f"Python版本:
        {python_version.major}.{python_version.minor}.{python_version.micro}")

    if python_version.major < 3 or (python_version.major == 3 and
        python_version.minor < 8):
        print(" 建议使用Python 3.8+版本")
```



```
print("\n检查必需库...")

# 必需库列表
required_packages = [
    ("numpy", "np"),
    ("pandas", "pd"),
    ("matplotlib.pyplot", "plt"),
    ("seaborn", "sns"),
    ("jieba", None),
    ("sklearn", None),
    ("jupyter", None)
]

success_count = 0
total_count = len(required_packages)

for package, alias in required_packages:
    if check_package(package, alias):
        success_count += 1

print("\n" + "=" * 50)
print(f"检查完成: {success_count}/{total_count} 库安装成功")

if success_count == total_count:
    print("    环境配置完美! 可以开始学习了!")

    # 测试基本功能
    print("\n测试基本功能...")
    test_basic_functions()
else:
    print("    部分库安装失败, 请检查安装命令")

def test_basic_functions():
    """测试基本功能是否正常"""
    try:
        import numpy as np
        import pandas as pd
```

```
import jieba

# 测试 numpy
arr = np.array([1, 2, 3, 4, 5])
print(f"NumPy 数组创建: {arr}")

# 测试 pandas
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
print(f"Pandas 数据框创建: \n{df}")

# 测试 jieba 分词
text = "我爱自然语言处理"
words = list(jieba.cut(text))
print(f"jieba 分词测试: {words}")

print(" 所有基本功能测试通过! ")

except Exception as e:
    print(f" 功能测试失败: {e}")

if __name__ == "__main__":
    main()
```

Listing 3: 环境验证脚本

3.2 2.2 核心库功能详解

表 7: NLP 核心库功能对比

库名称	主要功能	适用场景	核心特性
NumPy	数值计算、数组操作	矩阵运算、向量计算	高效的多维数组、广播机制、线性代数函数
Pandas	数据处理、分析	结构化数据操作	DataFrame 结构、数据清洗、统计分析
Matplotlib	基础图表绘制	静态可视化	丰富的图表类型、自定义样式、子图布局
Seaborn	统计图表美化	统计数据可视化	内置统计图表、美观默认样式、数据分布可视化
jieba	中文文本处理	中文分词、词性标注	多种分词模式、自定义词典、词性标注
scikit-learn	机器学习算法	特征工程、模型训练	丰富的算法库、标准化接口、模型评估
NLTK	英文 NLP 工具包	英文文本处理	语料库、分词器、词性标注、语法分析

3.3 2.3 Jupyter Notebook 高效使用指南 (15 分钟)

Jupyter Notebook 完全指南

启动 Jupyter

```
# 激活环境
conda activate nlp_course

# 启动 Jupyter Notebook
jupyter notebook

# 或启动 JupyterLab (推荐)
jupyter lab
```

核心快捷键

- Shift + Enter: 运行当前单元格并移动到下一个
- Ctrl + Enter: 运行当前单元格但不移动
- Alt + Enter: 运行当前单元格并在下方插入新单元格
- A: 在上方插入单元格 (命令模式)
- B: 在下方插入单元格 (命令模式)
- DD: 删除当前单元格 (命令模式)
- M: 转换为 Markdown 单元格
- Y: 转换为代码单元格

魔法命令 (Magic Commands)

- %matplotlib inline: 内嵌显示图片
- %time code: 计算单行代码执行时间
- %%time: 计算整个单元格执行时间
- %load file.py: 加载外部 Python 文件
- %who: 显示当前变量列表
- %whos: 显示详细变量信息
- %reset: 清空所有变量

4 第三部分：文本预处理实践 (120 分钟)

4.1 3.1 基础文本处理 (40 分钟)

4.1.1 文本清洗技术详解

文本清洗的重要性

为什么需要文本清洗？

- **噪声数据**：网络文本包含大量 HTML 标签、URL、表情符号
- **格式不一致**：空格、换行符、标点符号使用不规范
- **编码问题**：不同字符编码导致的乱码
- **无关信息**：对分析任务无意义的内容

清洗策略

1. **去除特殊字符**：HTML 标签、URL、邮箱地址
2. **标准化格式**：统一大小写、标点符号、空白字符
3. **编码规范化**：统一字符编码格式
4. **内容过滤**：删除过短或过长的文本

```
import re
import unicodedata
from typing import List, Optional

class AdvancedTextCleaner:
    """
    高级文本清洗器
    支持多种清洗策略和自定义规则
    """

    def __init__(self):
        """初始化清洗器，预编译正则表达式提高效率"""
        self.patterns = {
            # URL匹配 (包括http、https、ftp等)
            'url': re.compile(
```

```

        r'http[s]?://(?:[a-zA-Z]|[0-9]|[$-_@.&+]|[*\\(\)\,]|(?:%[0-9a-f
    ),
    # 邮箱地址匹配
    'email':
        re.compile(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b'
    # 电话号码匹配 (中国手机号)
    'phone': re.compile(r'1[3-9]\d{9}'),
    # HTML 标签匹配
    'html': re.compile(r'<[^>]+>'),
    # 多个空白字符
    'extra_space': re.compile(r'\s+'),
    # 特殊字符 (保留基本标点)
    'special_chars':
        re.compile(r'[^\\w\\s\\u4e00-\\u9fff, 。 ! ? ; : " ' ' ( ) 【 】 《 》 ]'),
    # 数字匹配
    'numbers': re.compile(r'\d+'),
    # 英文字符
    'english': re.compile(r'[a-zA-Z]+')
}

def remove_urls(self, text: str) -> str:
    """移除 URL 链接"""
    return self.patterns['url'].sub('', text)

def remove_emails(self, text: str) -> str:
    """移除邮箱地址"""
    return self.patterns['email'].sub('', text)

def remove_phone_numbers(self, text: str) -> str:
    """移除电话号码"""
    return self.patterns['phone'].sub('', text)

def remove_html_tags(self, text: str) -> str:
    """移除 HTML 标签"""
    return self.patterns['html'].sub('', text)

def normalize_whitespace(self, text: str) -> str:
    """标准化空白字符"""

```

```
        return self.patterns['extra_space'].sub(' ', text).strip()

def remove_special_chars(self, text: str, keep_chinese_punct:
    bool = True) -> str:
    """
    移除特殊字符
    Args:
        text: 输入文本
        keep_chinese_punct: 是否保留中文标点符号
    """
    if keep_chinese_punct:
        # 保留中文标点符号
        pattern =
            re.compile(r'[^\\w\\s\\u4e00-\\u9fff, 。! ? ; : ""' ' ( ) 【】 《》 ]')
    else:
        pattern = self.patterns['special_chars']
    return pattern.sub('', text)

def remove_numbers(self, text: str) -> str:
    """移除数字"""
    return self.patterns['numbers'].sub('', text)

def remove_english(self, text: str) -> str:
    """移除英文字符"""
    return self.patterns['english'].sub('', text)

def normalize_unicode(self, text: str) -> str:
    """Unicode标准化"""
    return unicodedata.normalize('NFKC', text)

def filter_by_length(self, text: str, min_length: int = 2,
    max_length: int = 1000) -> Optional[str]:
    """
    根据长度过滤文本
    Returns:
        过滤后的文本，如果不符合长度要求则返回None
    """
    if min_length <= len(text) <= max_length:
```



```
        return text
    return None

def clean_text(self, text: str, steps: List[str] = None) -> str:
    """
    综合文本清洗
    Args:
        text: 原始文本
        steps: 清洗步骤列表，默认执行所有步骤
    """
    if not text or not isinstance(text, str):
        return ""

    if steps is None:
        steps = ['unicode', 'html', 'url', 'email', 'phone',
                'special_chars', 'whitespace']

    cleaned_text = text

    # 执行指定的清洗步骤
    if 'unicode' in steps:
        cleaned_text = self.normalize_unicode(cleaned_text)
    if 'html' in steps:
        cleaned_text = self.remove_html_tags(cleaned_text)
    if 'url' in steps:
        cleaned_text = self.remove_urls(cleaned_text)
    if 'email' in steps:
        cleaned_text = self.remove_emails(cleaned_text)
    if 'phone' in steps:
        cleaned_text = self.remove_phone_numbers(cleaned_text)
    if 'special_chars' in steps:
        cleaned_text = self.remove_special_chars(cleaned_text)
    if 'numbers' in steps:
        cleaned_text = self.remove_numbers(cleaned_text)
    if 'english' in steps:
        cleaned_text = self.remove_english(cleaned_text)
    if 'whitespace' in steps:
        cleaned_text = self.normalize_whitespace(cleaned_text)
```

```
        return cleaned_text

def batch_clean(self, texts: List[str], steps: List[str] = None)
    -> List[str]:
    """批量清洗文本"""
    return [self.clean_text(text, steps) for text in texts]

def get_cleaning_stats(self, original_texts: List[str],
    cleaned_texts: List[str]) -> dict:
    """获取清洗统计信息"""
    original_lengths = [len(text) for text in original_texts]
    cleaned_lengths = [len(text) for text in cleaned_texts]

    return {
        'total_texts': len(original_texts),
        'avg_original_length': sum(original_lengths) /
            len(original_lengths),
        'avg_cleaned_length': sum(cleaned_lengths) /
            len(cleaned_lengths),
        'compression_ratio': sum(cleaned_lengths) /
            sum(original_lengths),
        'empty_after_cleaning': sum(1 for text in cleaned_texts
            if not text.strip())
    }

# 使用示例
if __name__ == "__main__":
    cleaner = AdvancedTextCleaner()

    # 测试文本
    sample_texts = [
        "访问我们的网站",
        "https://example.com, 联系邮箱: contact@example.com!!! ",
        "<p>这是HTML文本</p>, 包含<strong>标签</strong>",
        "我的电话号码是13812345678, 请联系我。",
        "自然语言处理 (NLP) 是AI的重要分支!!! @#$$",
        "    多余的    空格    需要处理    "
```

```
]

print("=== 文本清洗演示 ===")
for i, text in enumerate(sample_texts):
    cleaned = cleaner.clean_text(text)
    print(f"\n文本 {i+1}:")
    print(f"原文: {text}")
    print(f"清洗后: {cleaned}")

# 批量处理统计
cleaned_texts = cleaner.batch_clean(sample_texts)
stats = cleaner.get_cleaning_stats(sample_texts, cleaned_texts)

print("\n=== 清洗统计信息 ===")
for key, value in stats.items():
    print(f"{key}: {value:.3f}" if isinstance(value, float) else
          f"{key}: {value}")
```

Listing 4: 完整的文本清洗类实现

4.1.2 中文分词技术深度解析

中文分词的核心挑战

1. 分词歧义问题

- **交集型歧义**：“研究生物化学” → “研究/生物化学” vs “研究生/物化学”
- **组合型歧义**：“乒乓球拍卖” → “乒乓球拍/卖” vs “乒乓球/拍卖”
- **真歧义**：需要语境才能确定的分词方式

2. 未登录词识别

- **新词**：网络用语、专业术语
- **人名**：“欧阳修”、“司马迁”
- **地名**：“乌鲁木齐”、“呼和浩特”
- **机构名**：“中科院计算所”

3. 分词粒度选择

- **粗粒度**：适合主题分析
- **细粒度**：适合语法分析
- **应用相关**：根据下游任务需求调整

```
import jieba
import jieba.posseg as pseg
import jieba.analyse
from typing import List, Tuple, Dict, Set
import pickle
import os

class AdvancedChineseTokenizer:
    """
    高级中文分词器
    集成多种分词模式和自定义功能
    """

    def __init__(self, user_dict_path: str = None, stop_words_path:
```

```
str = None):
    """
    初始化分词器
    Args:
        user_dict_path: 用户自定义词典路径
        stop_words_path: 停用词文件路径
    """
    # 加载用户词典
    if user_dict_path and os.path.exists(user_dict_path):
        jieba.load_userdict(user_dict_path)
        print(f"已加载用户词典: {user_dict_path}")

    # 加载停用词
    self.stop_words = set()
    if stop_words_path and os.path.exists(stop_words_path):
        with open(stop_words_path, 'r', encoding='utf-8') as f:
            self.stop_words = set(f.read().splitlines())
        print(f"已加载停用词: {len(self.stop_words)} 个")

    # 初始化默认停用词
    self._init_default_stop_words()

    # 分词统计信息
    self.stats = {
        'total_texts': 0,
        'total_tokens': 0,
        'avg_tokens_per_text': 0,
        'unique_tokens': set()
    }

def _init_default_stop_words(self):
    """初始化默认停用词列表"""
    default_stop_words = {
        '的', '了', '在', '是', '我', '有', '和', '就', '不',
        '人',
        '都', '一', '一个', '上', '也', '很', '到', '说', '要',
        '去',
        '你', '会', '着', '没有', '看', '好', '自己', '这', '那',
```

```
        '什么', '怎么', '为什么', '因为', '所以', '但是', '如果',  
        '或者', '以及', '以上', '以下', '之前', '之后', '这样',  
        '那样', '这些', '那些', '这里', '那里'  
    }  
    self.stop_words.update(default_stop_words)  
  
    def precise_cut(self, text: str) -> List[str]:  
        """精确模式分词"""  
        return list(jieba.cut(text, cut_all=False))  
  
    def full_cut(self, text: str) -> List[str]:  
        """全模式分词"""  
        return list(jieba.cut(text, cut_all=True))  
  
    def search_cut(self, text: str) -> List[str]:  
        """搜索引擎模式分词"""  
        return list(jieba.cut_for_search(text))  
  
    def pos_tagging(self, text: str) -> List[Tuple[str, str]]:  
        """  
        词性标注  
        Returns:  
            List of (word, pos_tag) tuples  
        """  
        return [(word, pos) for word, pos in pseg.cut(text)]  
  
    def extract_keywords(self, text: str, topK: int = 10, method:  
        str = 'tfidf') -> List[Tuple[str, float]]:  
        """  
        关键词提取  
        Args:  
            text: 输入文本  
            topK: 返回关键词数量  
            method: 提取方法 ('tfidf' 或 'textrank')  
        """  
        if method == 'tfidf':  
            return jieba.analyse.extract_tags(text, topK=topK,  
                withWeight=True)
```

```
elif method == 'textrank':
    return jieba.analyse.textrank(text, topK=topK,
                                   withWeight=True)
else:
    raise ValueError("method must be 'tfidf' or 'textrank'")

def filter_by_pos(self, text: str, keep_pos: Set[str] = None) ->
List[str]:
    """
    根据词性过滤词汇
    Args:
        text: 输入文本
        keep_pos: 保留的词性集合，默认保留名词、动词、形容词
    """
    if keep_pos is None:
        keep_pos = {'n', 'nr', 'ns', 'nt', 'nw', 'nz', # 名词类
                    'v', 'vd', 'vn', 'vshi', 'vyou', 'vf', 'vx',
                    # 动词类
                    'a', 'ad', 'an', 'ag', 'al'} # 形容词类

    words_pos = self.pos_tagging(text)
    return [word for word, pos in words_pos if
            any(pos.startswith(p) for p in keep_pos)]

def remove_stop_words(self, words: List[str]) -> List[str]:
    """移除停用词"""
    return [word for word in words if word not in
            self.stop_words and len(word) > 1]

def add_user_word(self, word: str, freq: int = None, tag: str =
None):
    """添加用户自定义词汇"""
    if freq and tag:
        jieba.add_word(word, freq, tag)
    elif freq:
        jieba.add_word(word, freq)
    else:
        jieba.add_word(word)
```

```
print(f"已添加用户词汇: {word}")

def delete_user_word(self, word: str):
    """删除用户自定义词汇"""
    jieba.del_word(word)
    print(f"已删除用户词汇: {word}")

def suggest_freq(self, segment: str, tune: bool = False) -> int:
    """
    调节词汇频率
    Args:
        segment: 词汇或短语
        tune: 是否立即调节
    """
    freq = jieba.suggest_freq(segment, tune)
    print(f"词汇 '{segment}' 的建议频率: {freq}")
    return freq

def tokenize_with_options(self, text: str, options: Dict) ->
List[str]:
    """
    带选项的分词
    Args:
        options: 分词选项字典
            - mode: 分词模式 ('precise', 'full', 'search')
            - remove_stop_words: 是否移除停用词
            - filter_pos: 是否进行词性过滤
            - keep_pos: 保留的词性集合
            - min_length: 最小词长
    """
    # 选择分词模式
    mode = options.get('mode', 'precise')
    if mode == 'precise':
        words = self.precise_cut(text)
    elif mode == 'full':
        words = self.full_cut(text)
    elif mode == 'search':
        words = self.search_cut(text)
```



```
else:
    words = self.precise_cut(text)

# 词性过滤
if options.get('filter_pos', False):
    keep_pos = options.get('keep_pos', None)
    words = self.filter_by_pos(text, keep_pos)

# 移除停用词
if options.get('remove_stop_words', True):
    words = self.remove_stop_words(words)

# 长度过滤
min_length = options.get('min_length', 1)
words = [word for word in words if len(word) >= min_length]

# 更新统计信息
self._update_stats(words)

return words

def _update_stats(self, words: List[str]):
    """更新分词统计信息"""
    self.stats['total_texts'] += 1
    self.stats['total_tokens'] += len(words)
    self.stats['unique_tokens'].update(words)
    self.stats['avg_tokens_per_text'] = (
        self.stats['total_tokens'] / self.stats['total_texts']
    )

def get_tokenization_stats(self) -> Dict:
    """获取分词统计信息"""
    stats = self.stats.copy()
    stats['unique_tokens_count'] = len(stats['unique_tokens'])
    stats['vocabulary_size'] = len(stats['unique_tokens'])
    del stats['unique_tokens'] # 移除大集合以减少输出
    return stats
```

```
def save_user_dict(self, words: List[str], file_path: str):
    """保存用户词典"""
    with open(file_path, 'w', encoding='utf-8') as f:
        for word in words:
            f.write(f"{word}\n")
    print(f"已保存用户词典到: {file_path}")

def analyze_segmentation_quality(self, text: str) -> Dict:
    """
    分析分词质量
    Returns:
        包含各种质量指标的字典
    """
    precise_words = self.precise_cut(text)
    full_words = self.full_cut(text)
    search_words = self.search_cut(text)

    # 计算质量指标
    quality_metrics = {
        'text_length': len(text),
        'precise_tokens': len(precise_words),
        'full_tokens': len(full_words),
        'search_tokens': len(search_words),
        'avg_token_length': sum(len(w) for w in precise_words) /
            len(precise_words) if precise_words else 0,
        'single_char_ratio': sum(1 for w in precise_words if
            len(w) == 1) / len(precise_words) if precise_words
            else 0,
        'oov_estimation': sum(1 for w in precise_words if w not
            in jieba.dt.FREQ) / len(precise_words) if
            precise_words else 0
    }

    return quality_metrics

# 使用示例和测试
if __name__ == "__main__":
    # 创建分词器实例
```

```
tokenizer = AdvancedChineseTokenizer()

# 测试文本
test_texts = [
    "北京大学的学生正在学习自然语言处理技术",
    "机器学习和深度学习是人工智能的重要分支",
    "今天天气真好，我想去公园散步",
    "苹果公司发布了新的iPhone产品",
    "研究生物化学需要扎实的理论基础"
]

print("=== 分词模式对比 ===")
for text in test_texts[:2]: # 只显示前两个以节省空间
    print(f"\n原文: {text}")
    print(f"精确模式: {tokenizer.precise_cut(text)}")
    print(f"全模式: {tokenizer.full_cut(text)}")
    print(f"搜索模式: {tokenizer.search_cut(text)}")

print("\n=== 词性标注演示 ===")
sample_text = "张三在北京大学学习计算机科学"
pos_result = tokenizer.pos_tagging(sample_text)
for word, pos in pos_result:
    print(f"{word}/{pos}", end=" ")
print()

print("\n=== 关键词提取演示 ===")
long_text =
    "自然语言处理是计算机科学领域与人工智能领域中的一个重要方向。它研究能实现
keywords_tfidf = tokenizer.extract_keywords(long_text, topK=5,
    method='tfidf')
keywords_textrank = tokenizer.extract_keywords(long_text,
    topK=5, method='textrank')

print("TF-IDF 关键词:")
for word, weight in keywords_tfidf:
    print(f"    {word}: {weight:.4f}")

print("TextRank 关键词:")
```

Listing 5: 高级中文分词器实现

4.2 3.2 停用词处理与词频统计 (40 分钟)

4.3 3.3 综合文本处理流水线 (40 分钟)

4.3.1 端到端文本处理系统

文本处理流水线设计原则

模块化设计

- 单一职责：每个模块专注一个特定功能
- 可配置性：支持灵活的参数配置
- 可扩展性：便于添加新的处理步骤
- 错误处理：健壮异常处理机制

质量控制

- 输入验证：检查输入数据格式和质量
- 过程监控：记录每个步骤的处理效果
- 结果评估：量化评估处理质量
- 错误恢复：提供数据恢复机制

性能优化

- 批量处理：提高大规模数据处理效率
- 并行计算：利用多核处理能力
- 内存管理：优化内存使用
- 缓存机制：减少重复计算

```
import os
import json
import pickle
```

```
import logging
from pathlib import Path
from typing import List, Dict, Any, Optional, Callable, Union
from dataclasses import dataclass
from concurrent.futures import ThreadPoolExecutor,
    ProcessPoolExecutor
import time
from tqdm import tqdm

# 配置日志
logging.basicConfig(level=logging.INFO, format='%(asctime)s -
    %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

@dataclass
class ProcessingConfig:
    """文本处理配置类"""
    # 清洗配置
    clean_html: bool = True
    clean_urls: bool = True
    clean_emails: bool = True
    clean_phone: bool = True
    clean_special_chars: bool = True
    normalize_unicode: bool = True
    normalize_whitespace: bool = True

    # 分词配置
    tokenization_mode: str = 'precise' # 'precise', 'full', 'search'
    custom_dict_path: Optional[str] = None
    enable_pos_tagging: bool = False

    # 停用词配置
    remove_stopwords: bool = True
    stopwords_path: Optional[str] = None
    min_word_length: int = 2
    filter_by_pos: bool = False
    keep_pos_tags: List[str] = None
```

```
# 频率过滤配置
min_frequency: int = 1
max_frequency: Optional[int] = None
min_document_frequency: int = 1

# 输出配置
save_intermediate_results: bool = False
output_format: str = 'list' # 'list', 'string', 'dataframe'

# 性能配置
batch_size: int = 1000
n_workers: int = 4
use_multiprocessing: bool = False

class TextProcessingPipeline:
    """
    综合文本处理流水线
    集成清洗、分词、停用词过滤、频率分析等功能
    """

    def __init__(self, config: ProcessingConfig = None):
        """
        初始化处理流水线
        Args:
            config: 处理配置，如果为None则使用默认配置
        """
        self.config = config or ProcessingConfig()

        # 初始化各个组件
        self.text_cleaner = AdvancedTextCleaner()
        self.tokenizer = AdvancedChineseTokenizer(
            user_dict_path=self.config.custom_dict_path,
            stop_words_path=self.config.stopwords_path
        )
        self.stopwords_manager = IntelligentStopwordsManager()
        self.frequency_analyzer = AdvancedFrequencyAnalyzer()

        # 处理统计信息
```

```
self.processing_stats = {
    'total_texts': 0,
    'successful_processing': 0,
    'failed_processing': 0,
    'processing_time': 0,
    'average_processing_time': 0,
    'steps_completed': []
}

# 中间结果存储
self.intermediate_results = {}

logger.info("文本处理流水线初始化完成")

def process_single_text(self, text: str, text_id: Optional[str]
= None) -> Dict[str, Any]:
    """
    处理单个文本
    Args:
        text: 输入文本
        text_id: 文本标识符（可选）
    Returns:
        处理结果字典
    """
    start_time = time.time()
    result = {
        'text_id': text_id or f"text_{int(time.time())}",
        'original_text': text,
        'processing_steps': [],
        'success': False,
        'error_message': None
    }

    try:
        current_text = text

        # 步骤1: 文本清洗
        if any([self.config.clean_html, self.config.clean_urls,
```

```
        self.config.clean_emails, self.config.clean_phone,
        self.config.clean_special_chars,
        self.config.normalize_unicode,
        self.config.normalize_whitespace]):

    clean_steps = []
    if self.config.normalize_unicode:
        clean_steps.append('unicode')
    if self.config.clean_html: clean_steps.append('html')
    if self.config.clean_urls: clean_steps.append('url')
    if self.config.clean_emails:
        clean_steps.append('email')
    if self.config.clean_phone:
        clean_steps.append('phone')
    if self.config.clean_special_chars:
        clean_steps.append('special_chars')
    if self.config.normalize_whitespace:
        clean_steps.append('whitespace')

    cleaned_text =
        self.text_cleaner.clean_text(current_text,
        clean_steps)

    result['cleaned_text'] = cleaned_text
    result['processing_steps'].append('text_cleaning')
    current_text = cleaned_text

# 步骤2: 分词
tokenization_options = {
    'mode': self.config.tokenization_mode,
    'remove_stop_words': False, # 单独处理停用词
    'filter_pos': False,
    'min_length': 1
}

tokens =
    self.tokenizer.tokenize_with_options(current_text,
    tokenization_options)
```



```
result['tokens'] = tokens
result['processing_steps'].append('tokenization')

# 步骤3: 词性标注 (如果需要)
if self.config.enable_pos_tagging:
    pos_tags = self.tokenizer.pos_tagging(current_text)
    result['pos_tags'] = pos_tags
    result['processing_steps'].append('pos_tagging')

# 步骤4: 停用词过滤
if self.config.remove_stopwords:
    filtered_tokens =
        self.stopwords_manager.filter_stopwords(
            tokens, min_length=self.config.min_word_length
        )
    result['filtered_tokens'] = filtered_tokens
    result['processing_steps'].append('stopwords_filtering')
    tokens = filtered_tokens

# 步骤5: 词性过滤 (如果需要)
if self.config.filter_by_pos and
self.config.keep_pos_tags:
    pos_filtered_tokens = self.tokenizer.filter_by_pos(
        current_text, set(self.config.keep_pos_tags)
    )
    result['pos_filtered_tokens'] = pos_filtered_tokens
    result['processing_steps'].append('pos_filtering')
    tokens = pos_filtered_tokens

# 最终结果
result['final_tokens'] = tokens
result['token_count'] = len(tokens)
result['unique_token_count'] = len(set(tokens))
result['success'] = True

except Exception as e:
    result['error_message'] = str(e)
    logger.error(f"处理文本时出错: {e}")
```

```
# 记录处理时间
processing_time = time.time() - start_time
result['processing_time'] = processing_time

return result

def process_batch(self, texts: List[str], text_ids:
Optional[List[str]] = None) -> List[Dict[str, Any]]:
    """
    批量处理文本
    Args:
        texts: 文本列表
        text_ids: 文本标识符列表（可选）
    Returns:
        处理结果列表
    """
    if text_ids is None:
        text_ids = [f"text_{i}" for i in range(len(texts))]

    if len(texts) != len(text_ids):
        raise ValueError("texts和text_ids长度必须相同")

    start_time = time.time()
    results = []

    logger.info(f"开始批量处理 {len(texts)} 个文本")

    # 根据配置选择处理方式
    if self.config.use_multiprocessing and len(texts) >
        self.config.batch_size:
        results = self._process_parallel(texts, text_ids)
    else:
        results = self._process_sequential(texts, text_ids)

    # 更新统计信息
    total_time = time.time() - start_time
    successful = sum(1 for r in results if r['success'])
```

```
failed = len(results) - successful

self.processing_stats.update({
    'total_texts': self.processing_stats['total_texts'] +
        len(texts),
    'successful_processing':
        self.processing_stats['successful_processing'] +
        successful,
    'failed_processing':
        self.processing_stats['failed_processing'] + failed,
    'processing_time':
        self.processing_stats['processing_time'] + total_time,
    'average_processing_time': total_time / len(texts) if
        texts else 0
})

logger.info(f"批量处理完成：成功 {successful}，失败
            {failed}，总时间 {total_time:.2f}秒")

return results

def _process_sequential(self, texts: List[str], text_ids:
List[str]) -> List[Dict[str, Any]]:
    """顺序处理文本"""
    results = []

    for text, text_id in tqdm(zip(texts, text_ids),
        total=len(texts), desc="处理文本"):
        result = self.process_single_text(text, text_id)
        results.append(result)

        # 保存中间结果（如果需要）
        if self.config.save_intermediate_results:
            self.intermediate_results[text_id] = result

    return results

def _process_parallel(self, texts: List[str], text_ids:
```

```
List[str]) -> List[Dict[str, Any]]:
    """并行处理文本"""
    results = []

    # 分批处理
    batch_size = self.config.batch_size
    n_workers = self.config.n_workers

    with ThreadPoolExecutor(max_workers=n_workers) as executor:
        futures = []

        for i in range(0, len(texts), batch_size):
            batch_texts = texts[i:i+batch_size]
            batch_ids = text_ids[i:i+batch_size]

            future = executor.submit(self._process_sequential,
                                    batch_texts, batch_ids)
            futures.append(future)

        # 收集结果
        for future in tqdm(futures, desc="处理批次"):
            batch_results = future.result()
            results.extend(batch_results)

    return results

def analyze_processing_results(self, results: List[Dict[str, Any]]) -> Dict[str, Any]:
    """分析处理结果"""
    if not results:
        return {'error': '无处理结果'}

    successful_results = [r for r in results if r['success']]
    failed_results = [r for r in results if not r['success']]

    if not successful_results:
        return {'error': '所有文本处理失败'}
```

```
# 提取所有成功处理的词汇
all_tokens = []
documents = []

for result in successful_results:
    tokens = result.get('final_tokens', [])
    all_tokens.extend(tokens)
    documents.append(tokens)

# 进行频率分析
freq_stats =
    self.frequency_analyzer.analyze_corpus(documents)

# 计算处理效果统计
processing_analysis = {
    'total_texts': len(results),
    'successful_texts': len(successful_results),
    'failed_texts': len(failed_results),
    'success_rate': len(successful_results) / len(results),
    'average_tokens_per_text': sum(r.get('token_count', 0)
        for r in successful_results) /
        len(successful_results),
    'average_unique_tokens_per_text':
        sum(r.get('unique_token_count', 0) for r in
            successful_results) / len(successful_results),
    'total_processing_time': sum(r.get('processing_time', 0)
        for r in results),
    'average_processing_time_per_text':
        sum(r.get('processing_time', 0) for r in results) /
        len(results)
}

# 合并频率分析结果
analysis_result = {
    'processing_statistics': processing_analysis,
    'frequency_analysis': freq_stats,
    'pipeline_statistics': self.processing_stats
}
```

```
    return analysis_result

def save_results(self, results: List[Dict[str, Any]],
                 output_path: str,
                 format: str = 'json', include_intermediate: bool
                 = False):
    """
    保存处理结果
    Args:
        results: 处理结果列表
        output_path: 输出路径
        format: 输出格式 ('json', 'csv', 'pickle')
        include_intermediate: 是否包含中间结果
    """
    output    print("TextRank 关键词:")
for word, weight in keywords_textrank:
    print(f"  {word}: {weight:.4f}")

print("\n=== 分词质量分析 ===")
for text in test_texts[:3]:
    quality = tokenizer.analyze_segmentation_quality(text)
    print(f"\n文本: {text}")
    print(f"  平均词长: {quality['avg_token_length']:.2f}")
    print(f"  单字符词比例: {quality['single_char_ratio']:.2f}")
    print(f"  未登录词估计: {quality['oov_estimation']:.2f}")
```

Listing 6: 完整的文本处理流水线系统

4.3.2 停用词管理系统

停用词的作用机制

什么是停用词？停用词是在文本处理中通常被过滤掉的词汇，因为它们：

- **频率极高**：在几乎所有文档中都出现
- **语义价值低**：不携带特定的语义信息
- **噪声影响**：可能干扰文本分析结果

停用词类型

1. **功能词**：助词、介词、连词（的、了、在、和）
2. **代词**：人称代词、指示代词（我、你、这、那）
3. **语气词**：表达语气的词汇（啊、呢、吧）
4. **高频无义词**：根据特定领域定制

注意事项

- 不同任务对停用词的需求不同
- 情感分析中“不”、“很”等词可能很重要
- 需要根据具体应用场景调整停用词表

```
import json
import pickle
from collections import defaultdict, Counter
from typing import Set, List, Dict, Tuple
import numpy as np

class IntelligentStopwordsManager:
    """
    智能停用词管理系统
    支持动态停用词发现、领域自适应、效果评估
    """

    def __init__(self, language: str = 'chinese'):
```

```
初始化停用词管理器
Args:
    language: 语言类型, 支持 'chinese', 'english'
"""
self.language = language
self.stopwords = set()
self.custom_stopwords = set()
self.domain_stopwords = set()

# 统计信息
self.word_stats = defaultdict(lambda: {
    'total_freq': 0,
    'doc_freq': 0,
    'contexts': []
})

# 加载默认停用词
self._load_default_stopwords()

def _load_default_stopwords(self):
    """加载默认停用词表"""
    if self.language == 'chinese':
        chinese_stopwords = {
            # 助词类
            '的', '得', '地', '着', '了', '过', '起', '来',
            '去', '进', '出',
            # 介词类
            '在', '于', '从', '到', '向', '往', '朝', '沿',
            '顺', '逆',
            # 连词类
            '和', '或', '及', '以及', '并且', '而且', '但是',
            '可是', '然而',
            # 代词类
            '我', '你', '他', '她', '它', '我们', '你们',
            '他们', '这', '那',
            '这个', '那个', '这些', '那些', '这里', '那里',
            '这样', '那样',
            # 量词类
```



```
'一', '二', '三', '四', '五', '六', '七', '八',  
    '九', '十',  
    '个', '位', '名', '只', '条', '头', '匹', '张',  
    '间', '件',  
# 副词类  
    '很', '非常', '特别', '尤其', '比较', '相当',  
    '十分', '极其',  
    '也', '还', '都', '又', '再', '就', '才', '只',  
    '仅仅',  
# 疑问词  
    '什么', '怎么', '为什么', '哪个', '哪里', '哪些',  
    '多少',  
# 语气词  
    '啊', '呢', '吧', '哦', '嗯', '哈', '呵', '嘿',  
# 时间词（通用）  
    '今天', '明天', '昨天', '现在', '以前', '以后',  
    '之前', '之后',  
# 程度词  
    '比较', '更加', '最', '极', '超', '太', '挺', '够',  
# 其他高频词  
    '有', '是', '会', '能', '要', '可以', '应该',  
    '需要', '必须',  
    '说', '看', '想', '知道', '觉得', '认为', '表示',  
    '显示',  
    '因为', '所以', '如果', '虽然', '尽管', '无论',  
    '不管',  
    '关于', '对于', '根据', '按照', '通过', '经过',  
    '由于'  
}  
  
self.stopwords.update(chinese_stopwords)  
  
elif self.language == 'english':  
    english_stopwords = {  
        'a', 'an', 'and', 'are', 'as', 'at', 'be', 'by',  
        'for', 'from',  
        'has', 'he', 'in', 'is', 'it', 'its', 'of', 'on',  
        'that', 'the',  
        'to', 'was', 'were', 'will', 'with', 'the', 'this',
```

```
        'but', 'they',
        'have', 'had', 'what', 'said', 'each', 'which',
        'their', 'time',
        'would', 'there', 'we', 'when', 'your', 'can',
        'said', 'she',
        'her', 'him', 'one', 'our', 'out', 'day', 'get',
        'use', 'man',
        'new', 'now', 'way', 'may', 'say'
    }

    self.stopwords.update(english_stopwords)

def load_from_file(self, file_path: str, encoding: str =
    'utf-8'):
    """从文件加载停用词"""
    try:
        with open(file_path, 'r', encoding=encoding) as f:
            words = set(f.read().splitlines())
            self.custom_stopwords.update(words)
            self.stopwords.update(words)
            print(f"从文件 {file_path} 加载了 {len(words)}
                  个停用词")
    except FileNotFoundError:
        print(f"文件 {file_path} 不存在")
    except Exception as e:
        print(f"加载停用词文件时出错: {e}")

def save_to_file(self, file_path: str, stopword_type: str =
    'all'):
    """保存停用词到文件"""
    if stopword_type == 'all':
        words_to_save = self.stopwords
    elif stopword_type == 'custom':
        words_to_save = self.custom_stopwords
    elif stopword_type == 'domain':
        words_to_save = self.domain_stopwords
    else:
        raise ValueError("stopword_type must be 'all', 'custom',
                           or 'domain'")
```

```
with open(file_path, 'w', encoding='utf-8') as f:
    for word in sorted(words_to_save):
        f.write(f"{word}\n")
print(f"已保存 {len(words_to_save)}
      个{stopword_type}停用词到 {file_path}")

def add_words(self, words: List[str], word_type: str = 'custom'):
    """添加停用词"""
    if isinstance(words, str):
        words = [words]

    word_set = set(words)
    self.stopwords.update(word_set)

    if word_type == 'custom':
        self.custom_stopwords.update(word_set)
    elif word_type == 'domain':
        self.domain_stopwords.update(word_set)

    print(f"已添加 {len(word_set)} 个{word_type}停用词")

def remove_words(self, words: List[str]):
    """移除停用词"""
    if isinstance(words, str):
        words = [words]

    word_set = set(words)
    self.stopwords.difference_update(word_set)
    self.custom_stopwords.difference_update(word_set)
    self.domain_stopwords.difference_update(word_set)

    print(f"已移除 {len(word_set)} 个停用词")

def update_word_stats(self, documents: List[List[str]]):
    """更新词汇统计信息"""
    print("正在更新词汇统计信息...")
```

```
for doc_id, words in enumerate(documents):
    word_counter = Counter(words)
    unique_words = set(words)

    for word in unique_words:
        self.word_stats[word]['doc_freq'] += 1

    for word, freq in word_counter.items():
        self.word_stats[word]['total_freq'] += freq
    # 保存上下文 (前后词汇)
    for i, w in enumerate(words):
        if w == word:
            context = words[max(0, i-2):i] +
                words[i+1:min(len(words), i+3)]
            self.word_stats[word]['contexts'].append(context)

print(f"已更新 {len(self.word_stats)} 个词汇的统计信息")

def discover_stopwords(self, documents: List[List[str]],
                        freq_threshold: float = 0.8,
                        idf_threshold: float = 0.1) -> Set[str]:
    """
    自动发现停用词
    Args:
        documents: 文档列表，每个文档是词汇列表
        freq_threshold: 文档频率阈值 (出现在多少比例的文档中)
        idf_threshold: IDF阈值 (低于此值认为是停用词)
    """
    if not self.word_stats:
        self.update_word_stats(documents)

    total_docs = len(documents)
    candidate_stopwords = set()

    for word, stats in self.word_stats.items():
        # 计算文档频率比例
        doc_freq_ratio = stats['doc_freq'] / total_docs
```

```
# 计算IDF值
idf = np.log(total_docs / (stats['doc_freq'] + 1))

# 判断是否为停用词候选
if doc_freq_ratio >= freq_threshold and idf <=
    idf_threshold:
    candidate_stopwords.add(word)

print(f"发现 {len(candidate_stopwords)} 个停用词候选")
return candidate_stopwords

def evaluate_stopwords_effect(self, documents: List[List[str]],
                              labels: List[str] = None) -> Dict:
    """
    评估停用词过滤效果
    Returns:
        包含各种评估指标的字典
    """
    # 计算过滤前后的统计信息
    original_vocab = set()
    filtered_vocab = set()
    original_total_words = 0
    filtered_total_words = 0

    for words in documents:
        original_vocab.update(words)
        original_total_words += len(words)

        filtered_words = [w for w in words if w not in
                           self.stopwords]
        filtered_vocab.update(filtered_words)
        filtered_total_words += len(filtered_words)

    # 计算压缩比和其他指标
    vocab_compression = 1 - len(filtered_vocab) /
        len(original_vocab)
    text_compression = 1 - filtered_total_words /
        original_total_words
```

```
evaluation_results = {
    'original_vocabulary_size': len(original_vocab),
    'filtered_vocabulary_size': len(filtered_vocab),
    'vocabulary_compression_ratio': vocab_compression,
    'original_total_words': original_total_words,
    'filtered_total_words': filtered_total_words,
    'text_compression_ratio': text_compression,
    'stopwords_count': len(self.stopwords),
    'avg_doc_length_before': original_total_words /
        len(documents),
    'avg_doc_length_after': filtered_total_words /
        len(documents)
}

return evaluation_results

def filter_stopwords(self, words: List[str], min_length: int =
2) -> List[str]:
    """
    过滤停用词
    Args:
        words: 词汇列表
        min_length: 最小词长限制
    """
    return [word for word in words
            if word not in self.stopwords and len(word) >=
                min_length]

def get_stopwords_by_category(self) -> Dict[str, Set[str]]:
    """获取分类的停用词"""
    return {
        'default': self.stopwords - self.custom_stopwords -
            self.domain_stopwords,
        'custom': self.custom_stopwords,
        'domain': self.domain_stopwords,
        'all': self.stopwords
    }
```

```
def analyze_word_importance(self, word: str) -> Dict:
    """分析单个词汇的重要性"""
    if word not in self.word_stats:
        return {'error': '词汇不在统计数据中'}

    stats = self.word_stats[word]
    return {
        'word': word,
        'total_frequency': stats['total_freq'],
        'document_frequency': stats['doc_freq'],
        'is_stopword': word in self.stopwords,
        'contexts_sample': stats['contexts'][:5]  # 显示前5个上下文
    }

# 使用示例
if __name__ == "__main__":
    # 创建停用词管理器
    stopwords_manager = IntelligentStopwordsManager('chinese')

    # 示例文档（已分词）
    sample_documents = [
        ['我', '爱', '自然', '语言', '处理', '技术'],
        ['机器', '学习', '是', '人工智能', '的', '重要', '分支'],
        ['深度', '学习', '在', 'NLP', '领域', '应用', '广泛'],
        ['文本', '分类', '是', '常见', '的', 'NLP', '任务'],
        ['词', '向量', '可以', '表示', '词汇', '的', '语义', '信息']
    ]

    print("=== 停用词过滤演示 ===")
    original_words = ['我', '爱', '自然', '语言', '处理', '的', '技术']
    filtered_words =
        stopwords_manager.filter_stopwords(original_words)
    print(f"原词列表: {original_words}")
    print(f"过滤后: {filtered_words}")
```

```
print("\n=== 自动发现停用词 ===")
discovered =
    stopwords_manager.discover_stopwords(sample_documents,
                                         freq_threshold=0.6,
                                         idf_threshold=0.5)

print(f"发现的停用词候选: {discovered}")

print("\n=== 停用词效果评估 ===")
evaluation =
    stopwords_manager.evaluate_stopwords_effect(sample_documents)
for key, value in evaluation.items():
    if isinstance(value, float):
        print(f"{key}: {value:.3f}")
    else:
        print(f"{key}: {value}")
```

Listing 7: 智能停用词管理系统

4.3.3 深度词频分析系统

词频分析的数学基础

基本统计量

- 词频 (Term Frequency): $TF(t) = \frac{\text{count}(t)}{\text{total_words}}$
- 文档频率 (Document Frequency): $DF(t) = |\{d : t \in d\}|$
- 逆文档频率: $IDF(t) = \log \frac{|D|}{|\{d:t \in d\}|}$

分布规律

- Zipf 定律: 词频与排名成反比, $f_r \propto \frac{1}{r^\alpha}$
- Heaps 定律: 词汇量与文本长度的关系, $|V| = K \cdot n^\beta$
- 长尾分布: 少数高频词占据大部分词频

质量指标

- 词汇丰富度: $TTR = \frac{|\text{unique_words}|}{|\text{total_words}|}$
- hapax legomena: 只出现一次的词汇比例
- 熵值: $H = -\sum_i p_i \log p_i$, 衡量词汇分布的不确定性

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter, defaultdict
from typing import List, Dict, Tuple, Optional
from scipy import stats
import math

# 设置中文显示
plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS']
plt.rcParams['axes.unicode_minus'] = False

class AdvancedFrequencyAnalyzer:
```

高级词频分析系统

支持多维度统计分析、分布规律验证、可视化展示

"""

```
def __init__(self):
    """初始化分析器"""
    self.corpus_stats = {}
    self.word_frequencies = Counter()
    self.document_frequencies = defaultdict(int)
    self.word_positions = defaultdict(list)
    self.documents = []

def analyze_corpus(self, documents: List[List[str]]) -> Dict:
    """
    全面分析语料库
    Args:
        documents: 文档列表，每个文档是词汇列表
    Returns:
        综合统计信息字典
    """
    self.documents = documents
    self._compute_basic_statistics()
    self._compute_advanced_statistics()
    return self.get_comprehensive_stats()

def _compute_basic_statistics(self):
    """计算基础统计信息"""
    all_words = []
    unique_words_per_doc = set()

    for doc_id, words in enumerate(self.documents):
        all_words.extend(words)
        unique_doc_words = set(words)
        unique_words_per_doc.update(unique_doc_words)

    # 更新文档频率
    for word in unique_doc_words:
        self.document_frequencies[word] += 1
```

```
# 记录词汇位置
for pos, word in enumerate(words):
    self.word_positions[word].append((doc_id, pos))

# 更新词频统计
self.word_frequencies = Counter(all_words)

# 基础统计信息
self.corpus_stats.update({
    'total_documents': len(self.documents),
    'total_words': len(all_words),
    'unique_words': len(self.word_frequencies),
    'vocabulary_size': len(unique_words_per_doc),
    'avg_doc_length': len(all_words) / len(self.documents)
    if self.documents else 0,
    'max_doc_length': max(len(doc) for doc in
        self.documents) if self.documents else 0,
    'min_doc_length': min(len(doc) for doc in
        self.documents) if self.documents else 0
})

def _compute_advanced_statistics(self):
    """计算高级统计信息"""
    if not self.word_frequencies:
        return

    frequencies = list(self.word_frequencies.values())
    total_words = sum(frequencies)

    # 词汇丰富度指标
    type_token_ratio = self.corpus_stats['unique_words'] /
        self.corpus_stats['total_words']

    # Hapax legomena (只出现一次的词)
    hapax_count = sum(1 for freq in frequencies if freq == 1)
    hapax_ratio = hapax_count / len(frequencies)
```

```
# 词频分布统计
freq_counter = Counter(frequencies)

# 熵值计算
probabilities = [freq / total_words for freq in frequencies]
entropy = -sum(p * math.log2(p) for p in probabilities)

# Zipf分布拟合
zipf_params = self._fit_zipf_distribution()

# Heaps定律验证
heaps_params = self._verify_heaps_law()

self.corpus_stats.update({
    'type_token_ratio': type_token_ratio,
    'hapax_legomena_count': hapax_count,
    'hapax_legomena_ratio': hapax_ratio,
    'vocabulary_entropy': entropy,
    'most_frequent_word':
        self.word_frequencies.most_common(1)[0],
    'frequency_distribution': dict(freq_counter),
    'zipf_parameters': zipf_params,
    'heaps_parameters': heaps_params
})

def _fit_zipf_distribution(self) -> Dict:
    """拟合Zipf分布"""
    if len(self.word_frequencies) < 10:
        return {'error': '数据量不足'}

# 获取频率排序
frequencies = [freq for word, freq in
    self.word_frequencies.most_common()]
ranks = list(range(1, len(frequencies) + 1))

# 对数变换后进行线性拟合
log_ranks = [math.log(r) for r in ranks]
log_freqs = [math.log(f) for f in frequencies]
```

```
# 计算相关系数和拟合参数
slope, intercept, r_value, p_value, std_err =
    stats.linregress(log_ranks, log_freqs)

return {
    'alpha': -slope, # Zipf指数
    'constant': math.exp(intercept),
    'r_squared': r_value**2,
    'p_value': p_value,
    'standard_error': std_err
}

def _verify_heaps_law(self) -> Dict:
    """验证Heaps定律"""
    if len(self.documents) < 5:
        return {'error': '文档数量不足'}

# 计算累积词汇量
cumulative_words = 0
cumulative_vocab = set()
text_lengths = []
vocab_sizes = []

for doc in self.documents:
    cumulative_words += len(doc)
    cumulative_vocab.update(doc)
    text_lengths.append(cumulative_words)
    vocab_sizes.append(len(cumulative_vocab))

# 对数变换后线性拟合
if len(text_lengths) >= 3:
    log_lengths = [math.log(n) for n in text_lengths if n > 0]
    log_vocab_sizes = [math.log(v) for v in vocab_sizes if v > 0]

    if len(log_lengths) >= 3:
```

```
slope, intercept, r_value, p_value, std_err =
    stats.linregress(log_lengths, log_vocab_sizes)

    return {
        'beta': slope, # Heaps 指数
        'K': math.exp(intercept),
        'r_squared': r_value**2,
        'p_value': p_value,
        'standard_error': std_err
    }

return {'error': '无法拟合'}
```

```
def get_word_statistics(self, word: str) -> Dict:
    """获取单个词汇的详细统计信息"""
    if word not in self.word_frequencies:
        return {'error': f'词汇 "{word}" 不在语料库中'}

    total_words = self.corpus_stats['total_words']
    total_docs = self.corpus_stats['total_documents']

    frequency = self.word_frequencies[word]
    doc_freq = self.document_frequencies[word]
    positions = self.word_positions[word]

    # 计算各种指标
    tf = frequency / total_words
    df = doc_freq / total_docs
    idf = math.log(total_docs / doc_freq) if doc_freq > 0 else 0
    tf_idf = tf * idf

    # 计算位置分布
    position_variance = 0
    if len(positions) > 1:
        avg_position = sum(pos for doc_id, pos in positions) /
            len(positions)
        position_variance = sum((pos - avg_position)**2 for
            doc_id, pos in positions) / len(positions)
```

```
return {
    'word': word,
    'frequency': frequency,
    'relative_frequency': tf,
    'document_frequency': doc_freq,
    'document_frequency_ratio': df,
    'idf': idf,
    'tf_idf': tf_idf,
    'rank': sorted(self.word_frequencies.values(),
                    reverse=True).index(frequency) + 1,
    'appears_in_docs': doc_freq,
    'total_positions': len(positions),
    'position_variance': position_variance,
    'first_occurrence': min(positions)[1] if positions else
        -1,
    'last_occurrence': max(positions)[1] if positions else -1
}

def get_frequency_distribution_analysis(self) -> Dict:
    """分析词频分布特征"""
    frequencies = list(self.word_frequencies.values())

    if not frequencies:
        return {'error': '无词频数据'}

    freq_stats = {
        'mean': np.mean(frequencies),
        'median': np.median(frequencies),
        'std': np.std(frequencies),
        'min': min(frequencies),
        'max': max(frequencies),
        'percentile_25': np.percentile(frequencies, 25),
        'percentile_75': np.percentile(frequencies, 75),
        'skewness': stats.skew(frequencies),
        'kurtosis': stats.kurtosis(frequencies)
    }
```

```
# 频率分组分析
freq_groups = {
    'singletons': sum(1 for f in frequencies if f == 1),
    'low_freq_2_5': sum(1 for f in frequencies if 2 <= f <=
        5),
    'medium_freq_6_20': sum(1 for f in frequencies if 6 <= f
        <= 20),
    'high_freq_21_100': sum(1 for f in frequencies if 21 <=
        f <= 100),
    'very_high_freq_100plus': sum(1 for f in frequencies if
        f > 100)
}

return {
    'basic_statistics': freq_stats,
    'frequency_groups': freq_groups,
    'total_unique_words': len(frequencies)
}

def plot_frequency_distribution(self, top_n: int = 30, figsize:
    Tuple[int, int] = (15, 10)):
    """绘制词频分布图"""
    if not self.word_frequencies:
        print("没有词频数据可供可视化")
        return

    fig, axes = plt.subplots(2, 2, figsize=figsize)
    fig.suptitle('词频分布综合分析', fontsize=16,
        fontweight='bold')

    # 1. 高频词条形图
    top_words = self.word_frequencies.most_common(top_n)
    words, freqs = zip(*top_words)

    axes[0, 0].bar(range(len(words)), freqs, color='skyblue',
        alpha=0.7)
    axes[0, 0].set_title(f'Top {top_n} 高频词汇', fontsize=12)
    axes[0, 0].set_xlabel('词汇排名')
```



```
axes[0, 0].set_ylabel('词频')
axes[0, 0].tick_params(axis='x', rotation=45)

# 添加词汇标签（每隔几个显示一个）
step = max(1, len(words) // 10)
axes[0, 0].set_xticks(range(0, len(words),
                             step)\begin{table}[H]
output_path = Path(output_path)
output_path.parent.mkdir(parents=True, exist_ok=True)

if format.lower() == 'json':
    export_data = {
        'results': results,
        'processing_config': self.config.__dict__,
        'processing_statistics': self.processing_stats
    }

    if include_intermediate:
        export_data['intermediate_results'] =
            self.intermediate_results

    with open(output_path, 'w', encoding='utf-8') as f:
        json.dump(export_data, f, ensure_ascii=False,
                  indent=2)

elif format.lower() == 'csv':
    # 转换为扁平化的CSV格式
    flat_data = []
    for result in results:
        if result['success']:
            flat_data.append({
                'text_id': result['text_id'],
                'original_length':
                    len(result['original_text']),
                'token_count': result.get('token_count', 0),
                'unique_token_count':
                    result.get('unique_token_count', 0),
                'final_tokens': '
```

```
        '.join(result.get('final_tokens', [])),
        'processing_time':
            result.get('processing_time', 0),
        'processing_steps':
            ';'.join(result.get('processing_steps',
                                []))
    })

import pandas as pd
df = pd.DataFrame(flat_data)
df.to_csv(output_path, index=False, encoding='utf-8-sig')

elif format.lower() == 'pickle':
    with open(output_path, 'wb') as f:
        pickle.dump({
            'results': results,
            'config': self.config,
            'stats': self.processing_stats,
            'intermediate': self.intermediate_results if
                include_intermediate else None
        }, f)

else:
    raise ValueError("format must be 'json', 'csv', or
                      'pickle'")

logger.info(f"结果已保存到: {output_path}")

def load_results(self, file_path: str) -> Dict[str, Any]:
    """加载之前保存的处理结果"""
    file_path = Path(file_path)

    if not file_path.exists():
        raise FileNotFoundError(f"文件不存在: {file_path}")

    if file_path.suffix.lower() == '.json':
        with open(file_path, 'r', encoding='utf-8') as f:
            data = json.load(f)
```

```
elif file_path.suffix.lower() == '.pickle':
    with open(file_path, 'rb') as f:
        data = pickle.load(f)
else:
    raise ValueError("只支持加载 .json 或 .pickle
                      格式的文件")

logger.info(f"结果已从 {file_path} 加载")
return data

def create_processing_report(self, results: List[Dict[str,
Any]]) -> str:
    """创建详细的处理报告"""
    analysis = self.analyze_processing_results(results)

    if 'error' in analysis:
        return f"报告生成失败: {analysis['error']}"

    report = []
    report.append("=" * 80)
    report.append("文本处理流水线综合报告")
    report.append("=" * 80)

    # 处理统计信息
    proc_stats = analysis['processing_statistics']
    report.append(f"\n【处理统计信息】")
    report.append(f"总文本数: {proc_stats['total_texts']:,}")
    report.append(f"成功处理:
                  {proc_stats['successful_texts']:,}")
    report.append(f"处理失败: {proc_stats['failed_texts']:,}")
    report.append(f"成功率: {proc_stats['success_rate']:.2%}")
    report.append(f"平均处理时间:
                  {proc_stats['average_processing_time_per_text']:.4f}
                  秒/文本")
    report.append(f"总处理时间:
                  {proc_stats['total_processing_time']:.2f} 秒")

    # 词汇统计信息
```

```
report.append(f"\n【词汇统计信息】")
report.append(f"平均词汇数/文本：
    {proc_stats['average_tokens_per_text']:.2f}")
report.append(f"平均独特词汇数/文本：
    {proc_stats['average_unique_tokens_per_text']:.2f}")

# 频率分析信息
freq_stats = analysis['frequency_analysis']
report.append(f"\n【频率分析信息】")
report.append(f"总词汇数：{freq_stats.get('total_words',
    0):,}")
report.append(f"独特词汇数：{freq_stats.get('unique_words',
    0):,}")
report.append(f"词汇表大小：
    {freq_stats.get('vocabulary_size', 0):,}")
report.append(f"平均文档长度：
    {freq_stats.get('avg_doc_length', 0):.2f}")

if 'type_token_ratio' in freq_stats:
    report.append(f"类型-标记比：
        {freq_stats['type_token_ratio']:.4f}")
if 'vocabulary_entropy' in freq_stats:
    report.append(f"词汇熵值：
        {freq_stats['vocabulary_entropy']:.4f}")

# 流水线配置信息
report.append(f"\n【流水线配置】")
config_items = [
    ('分词模式', self.config.tokenization_mode),
    ('移除停用词', self.config.remove_stopwords),
    ('最小词长', self.config.min_word_length),
    ('清洗HTML', self.config.clean_html),
    ('清洗URL', self.config.clean_urls),
    ('标准化Unicode', self.config.normalize_unicode),
    ('批处理大小', self.config.batch_size),
    ('工作线程数', self.config.n_workers)
]
```

```
for name, value in config_items:
    report.append(f"{name}: {value}")

# 处理步骤统计
step_counts = {}
for result in results:
    if result['success']:
        for step in result.get('processing_steps', []):
            step_counts[step] = step_counts.get(step, 0) + 1

if step_counts:
    report.append(f"\n【处理步骤统计】")
    for step, count in step_counts.items():
        report.append(f"{step}: {count}, 次")

# 错误分析
failed_results = [r for r in results if not r['success']]
if failed_results:
    error_types = {}
    for result in failed_results:
        error = result.get('error_message', 'Unknown error')
        error_types[error] = error_types.get(error, 0) + 1

    report.append(f"\n【错误分析】")
    for error, count in error_types.items():
        report.append(f"{error}: {count} 次")

report.append("\n" + "=" * 80)

return "\n".join(report)

def optimize_pipeline(self, sample_texts: List[str],
target_metric: str = 'processing_time') -> Dict[str, Any]:
    """
    流水线性能优化
    Args:
        sample_texts: 样本文本用于测试
        target_metric: 优化目标指标
```

```
Returns:
    优化建议和结果
"""
logger.info("开始流水线性能优化分析...")

optimization_results = {
    'original_config': self.config.__dict__.copy(),
    'test_results': [],
    'recommendations': []
}

# 测试不同配置
test_configs = [
    # 基础配置
    {'name': 'current', 'changes': {}},
    # 关闭部分清洗步骤
    {'name': 'minimal_cleaning', 'changes': {
        'clean_emails': False, 'clean_phone': False
    }},
    # 调整分词模式
    {'name': 'search_mode', 'changes': {
        'tokenization_mode': 'search'
    }},
    # 调整批处理大小
    {'name': 'larger_batch', 'changes': {
        'batch_size': min(2000, len(sample_texts))
    }},
    # 启用多进程
    {'name': 'multiprocessing', 'changes': {
        'use_multiprocessing': True,
        'n_workers': min(8, os.cpu_count())
    }}
]

for test_config in test_configs:
    # 创建测试配置
    test_pipeline_config = ProcessingConfig(**{
        **self.config.__dict__,
```

```
        **test_config['changes']
    })

    # 创建临时流水线
    test_pipeline =
        TextProcessingPipeline(test_pipeline_config)

    # 执行测试
    start_time = time.time()
    test_results =
        test_pipeline.process_batch(sample_texts[:100]) #
        限制测试样本数量
    processing_time = time.time() - start_time

    # 计算指标
    successful_count = sum(1 for r in test_results if
        r['success'])
    avg_tokens = sum(r.get('token_count', 0) for r in
        test_results if r['success']) / max(successful_count,
        1)

    test_result = {
        'config_name': test_config['name'],
        'config_changes': test_config['changes'],
        'processing_time': processing_time,
        'success_rate': successful_count / len(test_results),
        'avg_tokens_per_text': avg_tokens,
        'throughput': len(sample_texts) / processing_time
    }

    optimization_results['test_results'].append(test_result)
    logger.info(f"测试配置 {test_config['name']}: 耗时
        {processing_time:.2f}秒")

    # 生成优化建议
    best_time = min(optimization_results['test_results'],
        key=lambda x: x['processing_time'])
    best_throughput = max(optimization_results['test_results'],
```

```
key=lambda x: x['throughput'])

optimization_results['recommendations'].append(
    f"最快处理速度: {best_time['config_name']} "
    f"(耗时 {best_time['processing_time']:.2f}秒)"
)
optimization_results['recommendations'].append(
    f"最高吞吐量: {best_throughput['config_name']} "
    f"({best_throughput['throughput']:.1f} 文本/秒)"
)

# 根据目标指标提供具体建议
if target_metric == 'processing_time':
    if best_time['config_name'] != 'current':
        optimization_results['recommendations'].append(
            f"建议采用 {best_time['config_name']} "
            f"配置以获得最佳处理速度"
        )
elif target_metric == 'throughput':
    if best_throughput['config_name'] != 'current':
        optimization_results['recommendations'].append(
            f"建议采用 {best_throughput['config_name']} "
            f"配置以获得最高吞吐量"
        )

logger.info("流水线优化分析完成")
return optimization_results

# 使用示例和演示
def demonstrate_pipeline():
    """演示文本处理流水线的完整功能"""

    print("=" * 80)
    print("文本处理流水线演示")
    print("=" * 80)

    # 1. 创建配置
    config = ProcessingConfig(
```



```
# 清洗配置
clean_html=True,
clean_urls=True,
clean_special_chars=True,
normalize_whitespace=True,

# 分词配置
tokenization_mode='precise',
enable_pos_tagging=False,

# 停用词配置
remove_stopwords=True,
min_word_length=2,

# 性能配置
batch_size=100,
n_workers=2,
use_multiprocessing=False
)

# 2. 创建流水线
pipeline = TextProcessingPipeline(config)

# 3. 准备测试数据
sample_texts = [
    "自然语言处理(NLP)是人工智能的重要分支，访问  

https://example.com 了解更多！",
    "<p>机器学习</p>和深度学习技术在NLP领域应用广泛，联系邮箱：test@example",
    "文本预处理包括分词、停用词过滤、词频统计等步骤",
    "中文分词是中文NLP的基础任务，具有很大的挑战性。。。",
    "词向量可以表示词汇的语义信息，支持语义相似度计算",
    "深度学习模型如BERT、GPT在自然语言理解任务中表现出色",
    "情感分析、文本分类、机器翻译都是常见的NLP应用",
    "文本处理流水线需要考虑效率、准确性和可扩展性",
    "停用词过滤可以减少噪声，提高文本分析的质量",
    "词频分析有助于理解文本的主题特征和内容分布"
]
```

```
# 4. 处理单个文本示例
print("\n【单文本处理示例】")
single_result = pipeline.process_single_text(sample_texts[0],
                                             "demo_text_1")
print(f"原文: {single_result['original_text']}")
print(f"清洗后: {single_result.get('cleaned_text', 'N/A')}")
print(f"分词结果: {single_result.get('tokens', [])}")
print(f"最终词汇: {single_result.get('final_tokens', [])}")
print(f"处理时间: {single_result.get('processing_time', 0):.4f}
      秒")
print(f"处理步骤: {'',
      '.join(single_result.get('processing_steps', []))}")

# 5. 批量处理示例
print(f"\n【批量处理 {len(sample_texts)} 个文本】")
batch_results = pipeline.process_batch(sample_texts)

# 6. 分析处理结果
print("\n【处理结果分析】")
analysis = pipeline.analyze_processing_results(batch_results)

proc_stats = analysis['processing_statistics']
print(f"成功率: {proc_stats['success_rate']:.2%}")
print(f"平均处理时间:
      {proc_stats['average_processing_time_per_text']:.4f} 秒/文本")
print(f"平均词汇数: {proc_stats['average_tokens_per_text']:.2f}")

# 7. 生成详细报告
print(f"\n【详细处理报告】")
report = pipeline.create_processing_report(batch_results)
print(report)

# 8. 保存结果
print(f"\n【保存处理结果】")
# pipeline.save_results(batch_results,
#                       'processing_results.json', 'json')
print("结果保存功能已配置 (示例中注释以避免文件操作)")
```

```
# 9. 性能优化分析
print(f"\n【性能优化分析】")
optimization = pipeline.optimize_pipeline(sample_texts[:5])  #
    使用小样本进行优化测试

print("优化建议:")
for recommendation in optimization['recommendations']:
    print(f"    • {recommendation}")

print("\n配置测试结果:")
for test_result in optimization['test_results']:
    print(f"    {test_result['config_name']}: "
          f"{test_result['processing_time']:.3f}秒, "
          f"{test_result['throughput']:.1f} 文本/秒")

print(f"\n{'='*80}")
print("流水线演示完成!")
print(f"\n{'='*80}")

if __name__ == "__main__":
    demonstrate_pipeline()
```

Listing 8: 高级词频分析系统

5 课后练习设计

练习目标与要求

通过课后练习，学员将：

- 巩固文本预处理的核心概念和技术
- 熟练运用 Python 进行实际的文本处理任务
- 培养数据分析和问题解决能力
- 建立完整的 NLP 项目开发经验

表 8: 课后练习分级设计

练习编号	练习名称	难度	预估时间	核心技能
练习 1	基础文本清洗项目		2-3 小时	文本清洗、分词对比
练习 2	词频分析与可视化		3-4 小时	统计分析、数据可视化
练习 3	停用词优化系统		4-5 小时	系统设计、性能优化
练习 4	领域文本分析项目		5-6 小时	综合应用、报告撰写

5.0.1 练习 1：基础文本清洗项目（ ）

练习 1：基础文本清洗项目

目标：掌握基本的文本清洗和分词技能

任务描述：

1. **数据收集：**收集 100 条微博、新闻标题或产品评论
2. **清洗实现：**实现自定义文本清洗函数，处理：
 - 移除 URL 链接和邮箱地址
 - 处理 HTML 标签和特殊符号
 - 标准化标点符号和空白字符
 - 处理表情符号（可选：转换为文字或移除）
3. **分词对比：**比较 jieba 的三种分词模式，分析各自特点
4. **质量评估：**设计指标评估清洗效果

提交要求：

- Python 代码文件（包含完整的清洗类实现）
- 处理前后的文本对比表格（Excel 或 CSV 格式）
- 分词模式对比分析报告（包含具体例子）
- 代码注释清晰，包含函数文档字符串

评估标准：

- **功能完整性 (40%)：**清洗功能是否齐全、正确
- **分析深度 (30%)：**对分词结果的分析是否深入
- **代码质量 (20%)：**代码结构、注释、可读性
- **创新性 (10%)：**是否有独特的处理方法或发现

扩展挑战：

- 实现自适应的清洗策略（根据文本类型调整）
- 设计文本质量评分算法
- 比较不同清洗策略对下游任务的影响

5.0.2 练习 2：词频分析与可视化 ()

练习 2：词频分析与可视化项目

目标：综合运用文本预处理和统计分析技能

任务描述：

1. **文学作品分析：**选择一部公版小说（如《西游记》、《三国演义》）

2. **预处理流程：**

- 文本分章节处理
- 完整的清洗和分词
- 自定义停用词表优化

3. **深度分析：**

- 词频分布分析（验证 Zipf 定律）
- 人物、地名、事件等实体频率统计
- 章节间词汇变化分析
- 情感词汇分布分析

4. **可视化展示：**

- 制作精美的词云图
- 绘制词频分布图和 Zipf 曲线
- 创建人物关系网络图（基于共现）
- 设计交互式图表展示

技术要求：

- 使用面向对象的程序设计
- 实现可配置的分析流水线
- 使用多种可视化库（matplotlib, seaborn, plotly 等）
- 生成 HTML 格式的交互式报告

提交内容：

- 完整的 Python 项目代码
- 数据分析的 Jupyter Notebook
- 可视化结果图表集合
- 文学作品文本分析报告（3000 字以上）

5.0.3 练习 3：智能停用词优化系统 ()

练习 3：智能停用词优化系统

目标：设计并实现智能化的停用词管理系统

核心任务：

- 1. 自动发现停用词：
 - 基于统计指标（TF-IDF、文档频率）
 - 基于信息论方法（熵值、互信息）
 - 基于机器学习方法（无监督聚类）
- 2. 领域适应性：
 - 不同领域的停用词自动识别
 - 动态停用词表更新机制
 - 停用词重要性评分系统
- 3. 效果评估：
 - 设计多维度评估指标
 - 实现 A/B 测试框架
 - 在实际任务中验证效果
- 4. 性能优化：
 - 大规模文本的高效处理
 - 内存优化和并行计算
 - 实时停用词更新算法

技术挑战：

- 实现高效的倒排索引
- 设计可扩展的系统架构
- 集成多种机器学习算法
- 构建实时监控和反馈机制

交付成果：

- 完整的停用词管理系统 (Python 包)
- 系统设计文档和 API 文档
- 性能测试报告和优化建议

5.0.4 练习 4：领域文本分析综合项目（ ）

练习 4：领域文本分析综合项目

目标：完成一个端到端的 NLP 应用项目

项目选择（任选其一）：

1. 社交媒体情感监控系统
2. 科技论文主题演化分析
3. 电商评论智能分析平台
4. 新闻文本自动摘要系统
5. 古典文学计算分析工具

项目要求：

1. 数据获取：

- 设计数据收集策略
- 实现网络爬虫（遵守 robots.txt）
- 数据质量控制和去重
- 构建结构化数据存储

2. 文本处理：

- 构建领域特定的预处理流水线
- 优化分词和实体识别效果
- 实现增量处理机制
- 建立处理质量监控体系

3. 深度分析：

- 实现多维度统计分析
- 应用高级 NLP 技术（如关键词提取、主题建模）
- 构建时间序列分析
- 开发预测或分类模型

4. 系统集成：

- 设计 RESTful API 接口
- 构建 Web 可视化界面
- 实现数据导出功能

技术栈建议：

6 技术栈概览

- 后端：Python, Flask/FastAPI, SQLite/PostgreSQL
- 前端：HTML/CSS/JavaScript, Chart.js/D3.js
- 数据处理：pandas, numpy, scikit-learn
- 可视化：matplotlib, plotly

7 Python 代码示例

以下是一个高级词频分析器的实现：

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter, defaultdict
from typing import List, Dict, Tuple, Optional, Any
import math
from scipy import stats

class AdvancedFrequencyAnalyzer:
    """高级词频分析器"""

    def __init__(self):
        self.word_frequencies = Counter()
        self.document_frequencies = defaultdict(int)
        self.documents = []
        self.corpus_stats = {}

    def add_document(self, document: List[str]) -> None:
        """添加文档到语料库"""
        if not document:
            return

        self.documents.append(document)
```

```
self.word_frequencies.update(document)

# 计算文档频率
unique_words = set(document)
for word in unique_words:
    self.document_frequencies[word] += 1

def analyze_corpus(self, documents: List[List[str]]) -> Dict:
    """分析整个语料库"""
    # 清空之前的数据
    self.word_frequencies.clear()
    self.document_frequencies.clear()
    self.documents.clear()

    # 添加所有文档
    for doc in documents:
        self.add_document(doc)

    # 计算基础统计信息
    total_words = sum(self.word_frequencies.values())
    unique_words = len(self.word_frequencies)
    total_documents = len(self.documents)

    # 文档长度统计
    doc_lengths = [len(doc) for doc in self.documents if doc]
    avg_doc_length = np.mean(doc_lengths) if doc_lengths else 0
    max_doc_length = max(doc_lengths) if doc_lengths else 0
    min_doc_length = min(doc_lengths) if doc_lengths else 0

    # 计算词汇丰富度指标
    type_token_ratio = unique_words / total_words if total_words
        > 0 else 0

    # 计算词汇熵
    word_probs = [freq/total_words for freq in
        self.word_frequencies.values()]
    vocabulary_entropy = -sum(p * math.log2(p) for p in
        word_probs if p > 0)
```

```
# Hapax legomena (只出现一次的词)
hapax_count = sum(1 for freq in
    self.word_frequencies.values() if freq == 1)
hapax_ratio = hapax_count / unique_words if unique_words > 0
    else 0

# 最高频词
most_frequent = self.word_frequencies.most_common(1)
most_frequent_word = most_frequent[0] if most_frequent else
    ("", 0)

# 存储统计信息
self.corpus_stats = {
    'total_documents': total_documents,
    'total_words': total_words,
    'unique_words': unique_words,
    'vocabulary_size': unique_words,
    'avg_doc_length': avg_doc_length,
    'max_doc_length': max_doc_length,
    'min_doc_length': min_doc_length,
    'type_token_ratio': type_token_ratio,
    'vocabulary_entropy': vocabulary_entropy,
    'hapax_legomena_count': hapax_count,
    'hapax_legomena_ratio': hapax_ratio,
    'most_frequent_word': most_frequent_word
}

# 计算 Zipf 分布参数
self._calculate_zipf_parameters()

# 计算 Heaps 定律参数
self._calculate_heaps_parameters()

return self.corpus_stats

def _calculate_zipf_parameters(self) -> None:
    """计算 Zipf 分布参数"""
```

```
if len(self.word_frequencies) < 2:
    return

frequencies = [freq for word, freq in
                self.word_frequencies.most_common()]
ranks = list(range(1, len(frequencies) + 1))

# 对数变换进行线性回归
log_ranks = np.log(ranks)
log_freqs = np.log(frequencies)

# 计算回归参数
slope, intercept, r_value, p_value, std_err =
    stats.linregress(log_ranks, log_freqs)

self.corpus_stats['zipf_parameters'] = {
    'alpha': -slope, # Zipf指数
    'constant': np.exp(intercept),
    'r_squared': r_value**2,
    'p_value': p_value,
    'std_error': std_err
}

def _calculate_heaps_parameters(self) -> None:
    """计算Heaps定律参数"""
    if len(self.documents) < 2:
        return

    cumulative_words = []
    cumulative_vocab = []
    vocab_set = set()
    word_count = 0

    for doc in self.documents:
        word_count += len(doc)
        vocab_set.update(doc)
        cumulative_words.append(word_count)
        cumulative_vocab.append(len(vocab_set))
```



```
if len(cumulative_words) < 2:
    return

# 对数变换进行线性回归
log_words = np.log(cumulative_words)
log_vocab = np.log(cumulative_vocab)

slope, intercept, r_value, p_value, std_err =
    stats.linregress(log_words, log_vocab)

self.corpus_stats['heaps_parameters'] = {
    'beta': slope, # Heaps 指数
    'K': np.exp(intercept),
    'r_squared': r_value**2,
    'p_value': p_value,
    'std_error': std_err
}

def get_word_statistics(self, word: str) -> Dict:
    """获取特定词汇的详细统计信息"""
    if word not in self.word_frequencies:
        return {'error': f'词汇 "{word}" 不在语料库中'}

    frequency = self.word_frequencies[word]
    total_words = sum(self.word_frequencies.values())
    total_docs = len(self.documents)

    # 计算各种统计指标
    relative_frequency = frequency / total_words
    document_frequency = self.document_frequencies[word]

    # IDF (Inverse Document Frequency)
    idf = math.log(total_docs / document_frequency) if
        document_frequency > 0 else 0

    # TF-IDF (简化版, 使用全局词频作为 TF)
    tf = frequency / total_words
```

```
tf_idf = tf * idf

# 排名
rank = None
for i, (w, f) in enumerate(self.word_frequencies.most_common(), 1):
    if w == word:
        rank = i
        break

return {
    'word': word,
    'frequency': frequency,
    'relative_frequency': relative_frequency,
    'document_frequency': document_frequency,
    'documents_containing': document_frequency,
    'idf': idf,
    'tf_idf': tf_idf,
    'rank': rank,
    'percentile': (1 - rank/len(self.word_frequencies)) *
        100 if rank else None
}

def get_frequency_distribution_analysis(self) -> Dict:
    """分析词频分布特征"""
    frequencies = list(self.word_frequencies.values())
    if not frequencies:
        return {}

    analysis = {
        'total_words': len(frequencies),
        'frequency_stats': {
            'mean': np.mean(frequencies),
            'median': np.median(frequencies),
            'std': np.std(frequencies),
            'min': min(frequencies),
            'max': max(frequencies),
            'q25': np.percentile(frequencies, 25),
```

```
        'q75': np.percentile(frequencies, 75)
    }
}

# 频率分组
frequency_groups = {
    'singletons': sum(1 for f in frequencies if f == 1),
    'low_freq_2_5': sum(1 for f in frequencies if 2 <= f <=
        5),
    'medium_freq_6_20': sum(1 for f in frequencies if 6 <= f
        <= 20),
    'high_freq_21_100': sum(1 for f in frequencies if 21 <=
        f <= 100),
    'very_high_freq_100plus': sum(1 for f in frequencies if
        f > 100)
}

analysis['frequency_groups'] = frequency_groups

return analysis

def plot_frequency_distribution(self, top_n: int = 50, figsize:
    Tuple[int, int] = (15, 10)) -> None:
    """绘制词频分布的多种可视化图表"""
    if not self.word_frequencies:
        print("没有词频数据，请先进行分析")
        return

    plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode
        MS']
    plt.rcParams['axes.unicode_minus'] = False

    fig, axes = plt.subplots(2, 2, figsize=figsize)
    fig.suptitle('词频分布综合分析', fontsize=16,
        fontweight='bold')

    # 1. 高频词条形图
    top_words = self.word_frequencies.most_common(min(top_n,
```

```
len(self.word_frequencies)))
words = [word for word, freq in top_words]
frequencies = [freq for word, freq in top_words]

bars = axes[0, 0].bar(range(len(words)), frequencies,
                      alpha=0.7, color='skyblue')
axes[0, 0].set_title(f'Top {len(words)} 高频词汇',
                    fontsize=12)
axes[0, 0].set_xlabel('词汇')
axes[0, 0].set_ylabel('频率')

# 添加词汇标签（每隔几个显示一个）
step = max(1, len(words) // 10)
axes[0, 0].set_xticks(range(0, len(words), step))
axes[0, 0].set_xticklabels([words[i] for i in range(0,
len(words), step)],
                           rotation=45, ha='right')

# 2. Zipf分布验证
frequencies_all = [freq for word, freq in
                   self.word_frequencies.most_common()]
ranks = list(range(1, len(frequencies_all) + 1))

axes[0, 1].loglog(ranks, frequencies_all, 'bo-', alpha=0.7,
                 markersize=3)
axes[0, 1].set_title('Zipf分布验证（对数-对数图）',
                    fontsize=12)
axes[0, 1].set_xlabel('词汇排名 (log)')
axes[0, 1].set_ylabel('词频 (log)')
axes[0, 1].grid(True, alpha=0.3)

# 添加拟合线
if 'zipf_parameters' in self.corpus_stats and 'alpha' in
    self.corpus_stats['zipf_parameters']:
    alpha = self.corpus_stats['zipf_parameters']['alpha']
    constant =
        self.corpus_stats['zipf_parameters']['constant']
    fitted_freqs = [constant / (r ** alpha) for r in ranks]
```

```
axes[0, 1].loglog(ranks, fitted_freqs, 'r--', alpha=0.8,
                  label=f'拟合线 ( $\alpha={\alpha:.2f}$ )')
axes[0, 1].legend()

# 3. 词频分布直方图
freq_values = list(self.word_frequencies.values())
axes[1, 0].hist(freq_values, bins=50, alpha=0.7,
                 color='lightgreen', edgecolor='black')
axes[1, 0].set_title('词频分布直方图', fontsize=12)
axes[1, 0].set_xlabel('词频')
axes[1, 0].set_ylabel('词汇数量')
axes[1, 0].set_yscale('log') # 使用对数尺度

# 4. 累积词汇量增长曲线 (Heaps定律)
cumulative_words = 0
cumulative_vocab = set()
text_lengths = []
vocab_sizes = []

for doc in self.documents:
    cumulative_words += len(doc)
    cumulative_vocab.update(doc)
    text_lengths.append(cumulative_words)
    vocab_sizes.append(len(cumulative_vocab))

axes[1, 1].plot(text_lengths, vocab_sizes, 'go-', alpha=0.7,
                 markersize=4)
axes[1, 1].set_title('词汇量增长曲线 (Heaps定律)',
                     fontsize=12)
axes[1, 1].set_xlabel('累积文本长度')
axes[1, 1].set_ylabel('累积词汇量')

# 添加Heaps定律拟合线
if 'heaps_parameters' in self.corpus_stats and 'beta' in
self.corpus_stats['heaps_parameters']:
    beta = self.corpus_stats['heaps_parameters']['beta']
    K = self.corpus_stats['heaps_parameters']['K']
    fitted_vocab = [K * (n ** beta) for n in text_lengths]
```

```
axes[1, 1].plot(text_lengths, fitted_vocab, 'r--',
                alpha=0.8,
                label=f'Heaps 拟合 ( $\beta = \{beta:.2f\}$ )')
axes[1, 1].legend()

plt.tight_layout()
plt.show()

def plot_word_cloud_data(self, max_words: int = 100) -> Dict:
    """准备词云数据"""
    # 过滤掉过短的词和数字
    filtered_freq = {word: freq for word, freq in
                     self.word_frequencies.items()
                     if len(word) >= 2 and not word.isdigit()}

    # 获取前 max_words 个词汇
    top_words =
        dict(Counter(filtered_freq).most_common(max_words))

    return {
        'word_frequencies': top_words,
        'max_frequency': max(top_words.values()) if top_words
        else 0,
        'min_frequency': min(top_words.values()) if top_words
        else 0,
        'total_words': len(top_words)
    }

def generate_frequency_report(self) -> str:
    """生成详细的词频分析报告"""
    if not self.corpus_stats:
        return "未进行词频分析，请先调用 analyze_corpus 方法"

    report = []
    report.append("=" * 60)
    report.append("词频分析综合报告")
    report.append("=" * 60)
```

```
# 基础统计信息
report.append("\n【基础统计信息】")
report.append(f"文档总数：
    {self.corpus_stats['total_documents'],}")
report.append(f"词汇总数：
    {self.corpus_stats['total_words'],}")
report.append(f"独特词汇数：
    {self.corpus_stats['unique_words'],}")
report.append(f"词汇表大小：
    {self.corpus_stats['vocabulary_size'],}")
report.append(f"平均文档长度：
    {self.corpus_stats['avg_doc_length']:.2f}")
report.append(f"最长文档：
    {self.corpus_stats['max_doc_length'],} 词")
report.append(f"最短文档：
    {self.corpus_stats['min_doc_length'],} 词")

# 词汇丰富度指标
report.append("\n【词汇丰富度指标】")
report.append(f"类型-标记比 (TTR):
    {self.corpus_stats['type_token_ratio']:.4f}")
report.append(f"词汇熵值：
    {self.corpus_stats['vocabulary_entropy']:.4f}")
report.append(f"Hapax legomena 数量：
    {self.corpus_stats['hapax_legomena_count'],}")
report.append(f"Hapax legomena 比例：
    {self.corpus_stats['hapax_legomena_ratio']:.4f}")

# 高频词信息
if 'most_frequent_word' in self.corpus_stats:
    word, freq = self.corpus_stats['most_frequent_word']
    report.append(f"\n【最高频词汇】")
    report.append(f"词汇： '{word}' (出现 {freq}, 次)")
    report.append(f"相对频率：
        {freq/self.corpus_stats['total_words']:.4f}")

# Zipf分布分析
if 'zipf_parameters' in self.corpus_stats and 'alpha' in
```

```
self.corpus_stats['zipf_parameters']:
    zipf = self.corpus_stats['zipf_parameters']
    report.append(f"\n【Zipf分布分析】 ")
    report.append(f"Zipf指数 ( ): {zipf['alpha']:.4f}")
    report.append(f"拟合优度 (R²): {zipf['r_squared']:.4f}")
    report.append(f"P值: {zipf['p_value']:.6f}")

# 解释Zipf指数
if zipf['alpha'] < 0.8:
    report.append(" → 词频分布比典型Zipf分布更平均")
elif zipf['alpha'] > 1.2:
    report.append(" → 词频分布比典型Zipf分布更集中")
else:
    report.append(" → 词频分布符合典型Zipf分布")

# Heaps定律分析
if 'heaps_parameters' in self.corpus_stats and 'beta' in
self.corpus_stats['heaps_parameters']:
    heaps = self.corpus_stats['heaps_parameters']
    report.append(f"\n【Heaps定律分析】 ")
    report.append(f"Heaps指数 ( ): {heaps['beta']:.4f}")
    report.append(f"拟合优度 (R²): {heaps['r_squared']:.4f}")
    report.append(f"P值: {heaps['p_value']:.6f}")

# 解释Heaps指数
if heaps['beta'] < 0.4:
    report.append(" → 词汇增长较慢, 文本重复性较高")
elif heaps['beta'] > 0.7:
    report.append(" → 词汇增长较快, 文本多样性较高")
else:
    report.append(" → 词汇增长符合典型模式")

# 频率分布分析
freq_analysis = self.get_frequency_distribution_analysis()
if 'frequency_groups' in freq_analysis:
    groups = freq_analysis['frequency_groups']
    report.append(f"\n【词频分组分析】 ")
    report.append(f"单次出现词汇: {groups['singletons'],,}
```



```
        ({groups['singletons']/self.corpus_stats['unique_words']*100:.1f})
    report.append(f"低频词汇 (2-5次):")
    {groups['low_freq_2_5'],})")
    report.append(f"中频词汇 (6-20次):")
    {groups['medium_freq_6_20'],})")
    report.append(f"高频词汇 (21-100次):")
    {groups['high_freq_21_100'],})")
    report.append(f"超高频词汇 (>100次):")
    {groups['very_high_freq_100plus'],})")

# Top词汇列表
report.append(f"\n【Top 20 高频词汇】")
top_20 = self.word_frequencies.most_common(20)
for i, (word, freq) in enumerate(top_20, 1):
    percentage = freq / self.corpus_stats['total_words'] *
        100
    report.append(f"{i:2d}. {word:<10} {freq:6,} 次
        ({percentage:5.2f}%)")

report.append("\n" + "=" * 60)

return "\n".join(report)

def get_comprehensive_stats(self) -> Dict:
    """获取综合统计信息"""
    return self.corpus_stats

def export_frequency_data(self, file_path: str, format: str =
    'csv'):
    """导出词频数据"""
    if format.lower() == 'csv':
        # 创建DataFrame
        data = []
        for word, freq in self.word_frequencies.most_common():
            word_stats = self.get_word_statistics(word)
            data.append({
                'word': word,
                'frequency': freq,
```

```
        'relative_frequency':
            word_stats['relative_frequency'],
        'document_frequency':
            word_stats['document_frequency'],
        'idf': word_stats['idf'],
        'tf_idf': word_stats['tf_idf'],
        'rank': word_stats['rank']
    })

df = pd.DataFrame(data)
df.to_csv(file_path, index=False, encoding='utf-8-sig')
print(f"词频数据已导出到: {file_path}")

elif format.lower() == 'json':
    import json
    export_data = {
        'corpus_statistics': self.corpus_stats,
        'word_frequencies': dict(self.word_frequencies),
        'document_frequencies':
            dict(self.document_frequencies)
    }
    with open(file_path, 'w', encoding='utf-8') as f:
        json.dump(export_data, f, ensure_ascii=False,
                  indent=2)
    print(f"词频数据已导出到: {file_path}")

else:
    raise ValueError("format must be 'csv' or 'json'")

# 使用示例和演示
if __name__ == "__main__":
    # 创建分析器实例
    analyzer = AdvancedFrequencyAnalyzer()

    # 准备示例数据（实际使用时应该是真实的分词结果）
    sample_documents = [
        ['自然', '语言', '处理', '是', '计算机', '科学', '的',
         '重要', '分支'],
```

```
['机器', '学习', '和', '深度', '学习', '是', '人工智能',  
  '的', '核心', '技术'],  
['文本', '分析', '需要', '先', '进行', '分词', '和', '词频',  
  '统计'],  
['词', '向量', '可以', '表示', '词汇', '的', '语义', '信息'],  
['自然', '语言', '处理', '在', '搜索', '引擎', '中', '应用',  
  '广泛'],  
['深度', '学习', '模型', '在', 'NLP', '任务', '中', '表现',  
  '优异'],  
['文本', '分类', '是', '常见', '的', '自然', '语言', '处理',  
  '任务'],  
['词频', '分析', '可以', '帮助', '理解', '文本', '的',  
  '主题', '特征'],  
['机器', '翻译', '需要', '大量', '的', '平行', '语料',  
  '训练'],  
['情感', '分析', '可以', '判断', '文本', '的', '情感',  
  '倾向']  
]  
  
print("开始词频分析...")  
  
# 执行综合分析  
comprehensive_stats = analyzer.analyze_corpus(sample_documents)  
  
# 打印分析报告  
print("\n" + analyzer.generate_frequency_report())  
  
# 词汇详细分析示例  
print("\n=== 具体词汇分析示例 ===")  
target_words = ['自然', '语言', '处理', '学习', '分析']  
for word in target_words:  
    stats = analyzer.get_word_statistics(word)  
    if 'error' not in stats:  
        print(f"\n词汇: {word}")  
        print(f"  频率: {stats['frequency']} 次")  
        print(f"  相对频率: {stats['relative_frequency']:.4f}")  
        print(f"  文档频率: {stats['document_frequency']}")  
        print(f"  排名: {stats['rank']}")
```

```
print(f"    TF-IDF: {stats['tf_idf']:.4f}")

# 可视化分析
print("\n正在生成可视化图表...")
analyzer.plot_frequency_distribution(top_n=20, figsize=(16, 12))

# 导出数据示例
print("\n导出分析结果...")
# analyzer.export_frequency_data('frequency_analysis.csv', 'csv')
# analyzer.export_frequency_data('frequency_analysis.json',
    'json')

print("\n词频分析完成！")
```

Listing 9: 高级词频分析器

```
# 1. 魔法命令演示
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np

# 2. 时间测量
%time result = sum(range(1000000))
print(f"计算结果: {result}")

# 3. 系统命令执行
!pip list | grep pandas # 查看pandas版本

# 4. 变量查看
x = [1, 2, 3, 4, 5]
y = np.array(x)
%whos # 查看所有变量详细信息

# 5. 代码性能分析
%%timeit
# 这个单元格的所有代码都会被多次执行以测量平均时间
total = 0
for i in range(1000):
    total += i * i
```

```
# 6. 调试模式
%debug # 在异常发生时启动调试器

# 7. 自动重载模块（开发时很有用）
%load_ext autoreload
%autoreload 2
```

Listing 10: Jupyter 高效使用技巧

最佳实践建议

1. 代码组织

- 每个单元格完成一个明确的功能
- 使用 Markdown 单元格添加说明文档
- 重要的常量和配置放在开头
- 函数定义单独放在一个单元格

2. 可视化设置

```
# 设置中文字体和图表样式
import matplotlib.pyplot as plt
import seaborn as sns

plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS']
plt.rcParams['axes.unicode_minus'] = False
sns.set_style("whitegrid")
plt.style.use('seaborn-v0_8-darkgrid')
```

3. 数据科学工作流

- 导入库 → 数据加载 → 数据探索 → 数据清洗 → 特征工程 → 建模 → 评估 → 可视化
- 每个步骤用 Markdown 标题分割
- 关键结果保存到变量，便于后续引用

8 总结与展望

8.1 课程总结

8.2 后续课程展望

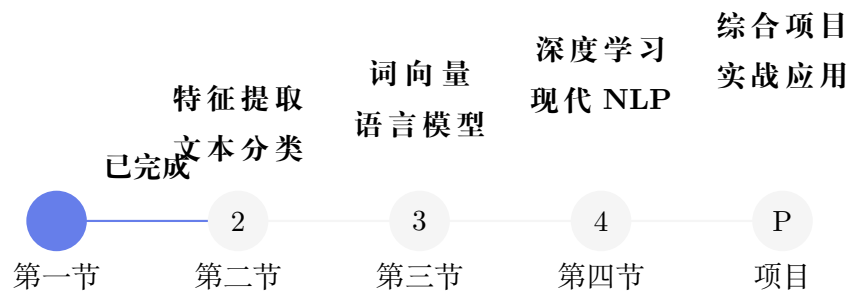


图 6: NLP 课程学习路线图

下节课预告：特征提取与文本分类

第二节课核心内容：

- **特征工程深化：** TF-IDF、N-gram 特征的高级应用
- **文本分类算法：** 朴素贝叶斯、SVM、逻辑回归的原理和实现
- **模型评估：** 交叉验证、混淆矩阵、ROC 曲线分析
- **实战项目：** 构建完整的文本分类系统

预习建议：

- 复习概率论和统计学基础知识
- 熟悉 scikit-learn 库的基本使用
- 了解机器学习的基本概念
- 完成本节课的所有练习作业

第一节课学习成果总结

通过本节课的学习，学员已经掌握了：

理论知识

- NLP 的基本概念、发展历程和应用领域
- 文本表示方法从符号化到分布式的演进
- 中文 NLP 的特点和主要技术挑战
- 文本预处理的重要性和核心技术

实践技能

- Python NLP 开发环境的搭建和配置
- 高质量文本清洗和标准化处理
- 中文分词技术的应用和优化
- 停用词管理和词频分析技术
- 完整文本处理流水线的设计和实现

工程能力

- 面向对象的程序设计思维
- 系统化的问题分析和解决方法
- 代码质量控制和文档规范
- 性能优化和错误处理机制

8.3 学习资源推荐

8.4 开发工具推荐

表 9: 开发工具推荐清单

工具类型	推荐工具	使用场景
IDE/编辑器	PyCharm, VS Code	日常开发和调试
交互式环境	Jupyter Lab, Google Colab	数据分析和实验
版本控制	Git, GitHub	代码管理和协作
包管理	conda, pip	环境和依赖管理
文档工具	Sphinx, MkDocs	API 文档生成
测试框架	pytest, unittest	代码测试和质量保证
性能分析	cProfile, line_profiler	代码性能优化
数据可视化	Tableau, Power BI	商业智能分析

9 附录

9.1 技术参考资料

- jieba 分词库文档: <https://github.com/fxsjy/jieba>
- NLTK 官方教程: <https://www.nltk.org/book/>
- pandas 用户指南: https://pandas.pydata.org/docs/user_guide/
- matplotlib 画廊: <https://matplotlib.org/stable/gallery/>
- 正则表达式教程: <https://regexr.com/>

9.2 常见问题解答 (FAQ)

1. Q: 中文分词效果不理想怎么办？

A: 可以尝试以下解决方案：

- 添加领域专用词典
- 调整分词模式（精确/全模式/搜索模式）
- 使用其他分词工具（LTP、pkuseg、Stanza）
- 基于字符的处理方法

- 后处理纠错机制

2. Q: 处理大规模文本时内存不足？

A: 内存优化策略：

- 使用生成器进行流式处理
- 分批处理大文件
- 使用更高效的数据结构
- 启用垃圾回收机制
- 考虑使用分布式处理

3. Q: 如何评估文本预处理的质量？

A: 评估方法包括：

- 人工抽样检查
- 统计指标分析（词汇丰富度、长度分布）
- 下游任务效果验证
- A/B 测试比较
- 错误率分析

4. Q: 停用词表应该如何选择？

A: 选择原则：

- 根据应用场景定制
- 考虑领域特殊性
- 平衡覆盖率和精确率
- 定期更新和维护
- 结合自动发现机制

9.3 持续学习建议

学习方法论指导

建立学习习惯

- **每日编程**：保持每天至少 1 小时的编程练习
- **项目驱动**：通过实际项目巩固理论知识
- **社区参与**：加入 NLP 技术社区，参与讨论
- **论文阅读**：定期阅读最新的研究论文

技能发展路径

1. 基础巩固期 (1-2 个月):

- 熟练掌握 Python 和相关库
- 完成所有课程练习
- 参与 1-2 个小型项目

2. 技能提升期 (3-6 个月):

- 学习深度学习框架
- 参与 Kaggle 竞赛
- 贡献开源项目

3. 专业发展期 (6 个月以上):

- 专注特定领域深入研究
- 发表技术文章或论文
- 指导其他学习者

资源管理

- **时间管理**：制定明确的学习计划和时间表
- **资料整理**：建立个人知识库和代码仓库
- **网络建设**：建立技术交流和合作网络
- **持续更新**：跟踪技术发展，更新知识结构

表 10: 分层次学习资源推荐

资源类型	初学者	进阶者	专家级
书籍	《Python 自然语言处理》 《统计自然语言处理基础》	《深度学习进阶：自然语言处理》 《Speech and Language Processing》	《Foundations of Statistical NLP》 最新顶会论文集
在线课程	CS224N (Stanford) Coursera NLP 专项课程	CS224U (Stanford) 深度学习专项课程	最新 Workshop 视频 专家讲座系列
实践平台	Kaggle Learn GitHub 入门项目	Kaggle 竞赛 开源项目贡献	研究项目领导 技术分享演讲
工具库	jieba, NLTK pandas, matplotlib	spaCy, gensim scikit-learn	Transformers 自主开发工具